

RESEARCH ARTICLE

Persistent Goal-Tracking and Instruction-Driven Reasoning in Sequential Conversation QA

QUOC-DUNG NGO^{ID}, QUANG-DUNG DANG^{ID}, AND LY-HUYNH PHAN^{ID}

Data Governance Laboratory, Research Institute of Posts and Telecommunications, Posts and Telecommunications Institute of Technology, Hanoi 122300, Vietnam

Corresponding author: Ly-Huynh Phan (huynhpl@ptit.edu.vn)

ABSTRACT Traditional conversational Question Answering (QA) systems focus on isolated information retrieval and lack the ability to maintain goal persistence across multi-turn interactions. Current Retrieval-Augmented Generation (RAG) approaches operate passively, retrieving information turn-by-turn without tracking whether the overall task objective has been achieved, and are typically evaluated using linguistic similarity metrics rather than functional task outcomes. We introduce **GSC-QA** (Goal-based Sequential Conversation QA), a framework that integrates three complementary components into a unified enterprise dialogue architecture: 1) persistent Goal Tracking Module (GTM) that maintains task state across turns in external databases (Redis + PostgreSQL), decoupled from the LLM context window; 2) Expert Instruction Layer that injects domain-curated procedural constraints into every response generation step; and 3) proactive slot detection that identifies and elicits missing required information before proceeding. Unlike prior task-oriented systems that assume static ontologies or session-scoped memory, **GSC-QA** combines retrieval, instruction enforcement, and goal persistence in a single coordinated loop built on LangGraph. Evaluated on 1,516 annotated dialogue records in Vietnamese spanning ten enterprise domains (principally beauty services, professional services, fashion & retail, spa & wellness, and language education, with smaller deployments in marketing, healthcare, banking, and lighting & electrical equipment), **GSC-QA** achieves a Task Success Rate (TSR) of 87.1% and a Goal Consistency Score (GCS) of 0.78 on the combined 1,516-record evaluation set, and an Error Recovery Rate (ERR) of 79.5% on 205 organic pivot transitions, substantially outperforming standard RAG (58.7% TSR, GCS 0.62). All baselines including Instruct-Only are evaluated on the same dataset for fair comparison. Cross-domain evaluation across 980 records from 14 enterprise deployments shows the mechanism transfers with little loss: on a curated 299-record subset scored under the same judge and rubric as the primary set, task success reaches 62.2% against 63.2% in-domain. Against four external baselines, including an agentic ReAct system, a memory-augmented long-context model, a SimpleTOD-style tracker and a graph-structured stateful agent, **GSC-QA** leads on task success, faithfulness and answer relevancy alike, and its lead in task success holds under four independent judge families. An LLM-as-judge metric (gpt-5.4) achieves 94% agreement with human annotations. Ablation studies confirm that goal persistence and expert instructions are complementary; removing either component causes significant performance degradation.

INDEX TERMS Sequential dialogue, task-oriented dialogue, retrieval-augmented generation, goal persistence, expert instructions.

I. INTRODUCTION

A. CONTEXT

Recent advances in large language models (LLMs), including GPT-3.5, GPT-4, and LLaMA variants, have driven the rapid

proliferation of conversational question answering (QA) systems across education, healthcare, business automation, and mental health services [1]. In enterprise contexts, conversational AI is shifting from passive inquiry resolution toward the active fulfilment of complex user objectives through sequential dialogue and sustained situational awareness [2]. This transition demands systems capable of combining intent

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Wang^{ID}.

recognition with personalized knowledge retrieval, automating multi-step professional workflows [3], and proactively steering users toward successful task outcomes [4], thereby repositioning conversational agents from reactive retrieval interfaces to strategic operational partners within enterprise productivity ecosystems [5].

B. RELATED WORKS

Substantial research has addressed multi-turn conversational QA from multiple directions, though traditional systems continue to struggle with coreference resolution, implicit queries, topic switching, and evolving user intent [6], [7], [8]. Growing attention to task-centric and action-oriented evaluation further broadens enterprise chatbot assessment beyond text similarity [9].

Among the earliest and most foundational of these directions, **task-oriented dialogue** systems have progressively advanced the challenge of maintaining conversational context across multiple turns [10]. Dialog State Tracking (DST) established the critical mechanism for tracking user intent and slot values [11], upon which TRADE [12] introduced copy mechanisms for flexible cross-domain state generation and SimpleTOD [13] unified DST, policy learning, and response generation into a single autoregressive framework. More recently, multi-agent architectures [14] and hybrid transformer-based frameworks [15] have further improved task decomposition and intent classification, particularly in few-shot enterprise deployment scenarios.

While task-oriented systems address context maintenance, a parallel stream recognizes that effective enterprise agents must go further by **proactively** leading conversations toward **predefined targets** rather than passively awaiting user queries. In [5], authors establish foundational concepts for conversation steering and target-driven planning, while the MultiWOZ benchmark family, successively refined through annotation corrections and co-reference resolution [16], has become the standard ground for evaluating slot-level dialogue state tracking. However, these approaches treat proactive management as a standalone component decoupled from retrieval and knowledge grounding, limiting their applicability in enterprise deployments where proactive clarification, retrieval, and instruction enforcement must operate in tight coordination.

Recognizing the limitations of purely parametric knowledge in LLMs, **retrieval-augmented generation (RAG)** [17] has emerged as the dominant paradigm for grounding LLM responses in external knowledge, substantially reducing factual hallucination and improving domain fidelity [18], [19]. Recent advances have extended RAG toward multi-turn settings through conversation-level augmentation [20], context-aware retrieval decisions [7], and hybrid sparse-dense strategies that often outperform more complex architectures [6]. Nevertheless, standard RAG retrieves informational content without enforcing procedural constraints on generation, a critical limitation in enterprise workflows requiring deterministic, sequenced operations.

Complementing retrieval-augmented approaches, **agentic AI systems** have demonstrated the capacity to autonomously interface with external systems through structured tool invocation. Toolformer [21] demonstrated that models can learn when and how to call external APIs through self-supervised training, while subsequent tool learning frameworks [22] broadened available invocation strategies. Building upon these capabilities, ReAct [23] and Reflexion [24] further demonstrated sustained multi-step task execution through interleaved reasoning-action cycles and iterative self-correction. Yet these frameworks remain oriented toward open-ended tool discovery without mechanisms for constraining agent behavior within enterprise-specific procedural requirements, a significant gap in regulated enterprise environments.

Despite this progress, **evaluation methodologies** for conversational AI systems have struggled to keep pace. Bilingual Evaluation Understudy (BLEU) and Recall-Oriented Understudy for Gisting Evaluation (ROUGE) were shown to correlate poorly with human judgments [25], motivating reference-free alternatives such as Unreferenced Sentence Representation (USR) [26] that assess coherence and groundedness without gold-standard references. Taking this further, action-oriented frameworks such as ABCD [27] measure task completion through agent actions and policy compliance rather than text similarity, providing operationally meaningful assessments aligned with enterprise deployment criteria [9], [27].

C. THE NECESSITY OF GOAL-BASED SEQUENTIAL CONVERSATION QA FRAMEWORK

Despite these advances, no existing framework simultaneously addresses goal persistence, procedural enforcement, proactive slot management, and task-centric evaluation within a unified enterprise architecture. Specifically, the following critical limitations persist:

- Task-oriented approaches assume static ontologies with predictable user trajectories, lacking mechanisms for goal persistence under arbitrary tangents or implicit slot references [11].
- Proactive dialogue approaches treat missing-slot detection as a standalone problem decoupled from retrieval [5], while agentic frameworks prioritize open-ended tool discovery over enterprise-specific procedural compliance [23], [24].
- Current systems adopt a passive interaction style, awaiting explicit user input rather than proactively guiding users through multi-step operational logic [14].

Explicit dialogue state maintained outside the language model is the defining idea of information-state and graph-based dialogue management, realised in the TRINDI information-state approach [28] and in RavenClaw's task-tree agenda [29], and given a probabilistic treatment in POMDP-based spoken dialogue systems [30]. Schema-guided dialogue [31] contributed the idea that goals and

slots should be declared as data rather than compiled into the model, and dataflow synthesis [32] showed that dialogue can be represented as an explicit, inspectable program state. End-to-end neural task-oriented systems, including SimpleTOD [13], SOLOIST [33], UBAR [34], and PPTOD [35], took the opposite route and folded belief tracking into the generative model itself. In the LLM era, MemGPT [36] reintroduced externalised, paged memory for long-horizon context; ReAct [23] and Reflexion [24] established interleaved reasoning and acting; and orchestration frameworks such as LangGraph [37] together with tool-interoperability standards such as the Model Context Protocol [38] have made graph-structured, stateful agents the prevailing commercial engineering pattern. On the evaluation side, CoQA [39] and QuAC [40] established conversational QA as a benchmark task, but score answer spans rather than whether a business goal was ultimately completed.

Our contribution is therefore one of integration and enterprise operationalization. Along the four axes that matter for the deployments we study, no prior paradigm supplies all four: information-state and schema-guided systems declare a goal and slot ontology but predate operator-authored procedural instruction and retrieval grounding; end-to-end neural systems fold state into the model and so externalise none of it; and memory-augmented and graph-orchestrated LLM agents persist state outside the model but leave its schema free-form, with no declared required-slot set against which completion is defined. What distinguishes GSC-QA is not that it stores state, but that a schema-constrained goal and slot ontology, operator-authored procedural instructions, proactive elicitation of missing slots, and retrieval grounding are made to operate over the same persistent state within one production system, and that the combination is evaluated on task completion in live commercial traffic rather than on span overlap or corpus-level belief accuracy. Section IV tests this claim directly, reporting a graph-structured stateful agent as an external baseline: it persists state, follows the shared instructions, and still completes far fewer tasks, which is the evidence that the schema rather than the persistence carries the effect.

In summary, these gaps expose the absence of a conversational AI framework capable of simultaneously maintaining goal awareness, enforcing procedural correctness, and enabling proactive task collaboration across sequential multi-turn enterprise dialogues.

D. OBJECTIVES

To bridge the gap between reactive language models and proactive task-completion agents, we propose the **Goal-based Sequential Conversation QA (GSC-QA)** framework, integrating expert instructions, task-persistence, and persona-driven constraints to improve multi-turn task completion reliability, procedural adherence, and goal consistency in enterprise conversational systems. The main contributions of this work are as follows:

- 1) **Sequential Goal Logic:** A conversation orchestration topology derived from LangGraph [37] that embeds business-specific goals across temporal sessions using bifurcated database architectures (Redis for short-term context; PostgreSQL for long-term goal specifications), enabling persistent goal tracking independent of LLM context window limitations.
- 2) **Expert Instruction Layer:** A methodology for integrating domain-curated structured instructions directly into the inference layer alongside semantic retrieval, constraining the LLM to follow deterministic procedural logic and preventing hallucinated or procedurally non-compliant workflow steps.
- 3) **Task-Centric Evaluation:** An outcome-focused evaluation framework that prioritizes goal fulfillment - operationalized through API execution completion and slot fulfillment rates - over surface-level text overlap metrics [25], supplemented by LLM-as-judge goal consistency assessment validated against human annotation.

The remainder of this paper is organized as follows. Section II presents the GSC-QA architecture and its four core operational modules in detail. Section III details the experimental dataset construction, evaluation baselines, and assessment metrics. Section IV reports quantitative results, ablation analyses, per-goal performance breakdowns, and LLM backbone comparisons. Section V summarizes the principal contributions and outlines promising directions for future research.

II. PROPOSED METHODOLOGY: THE GSC-QA FRAMEWORK

A. PROBLEM FORMULATION

We define a goal-directed dialogue as a sequence of states $S = \{s_1, s_2, \dots, s_T\}$ where each state is a tuple:

$$s_t = (u_t, h_t, g, M_{lt}, M_{st}) \quad (1)$$

where u_t is the user utterance at turn t , h_t is the dialogue history, $g = (\text{goal_id}, R, I)$ is the goal specification comprising an identifier, a set of required slots $R = \{r_1, \dots, r_k\}$, and expert instructions I . M_{lt} and M_{st} denote long-term and short-term memory respectively.

Definition 1 (Task Completion): A dialogue $D = \{s_1, \dots, s_T\}$ achieves task completion if and only if:

$$\forall r_i \in R : \text{filled}(r_i) = \text{true} \wedge \text{action}(g) \text{ executed} \quad (2)$$

B. ARCHITECTURE OVERVIEW

The framework comprises four core modules operating through a hub-and-spoke state machine (Fig. 1):

- **Goal Tracking Module (GTM):** A persistent validation node that orchestrates long-term conversational focus by maintaining the target state in external databases, including session authentication and state management.
- **Retrieval-Augmented Knowledge Module (RAKM):** Augmentation middleware that combines hybrid

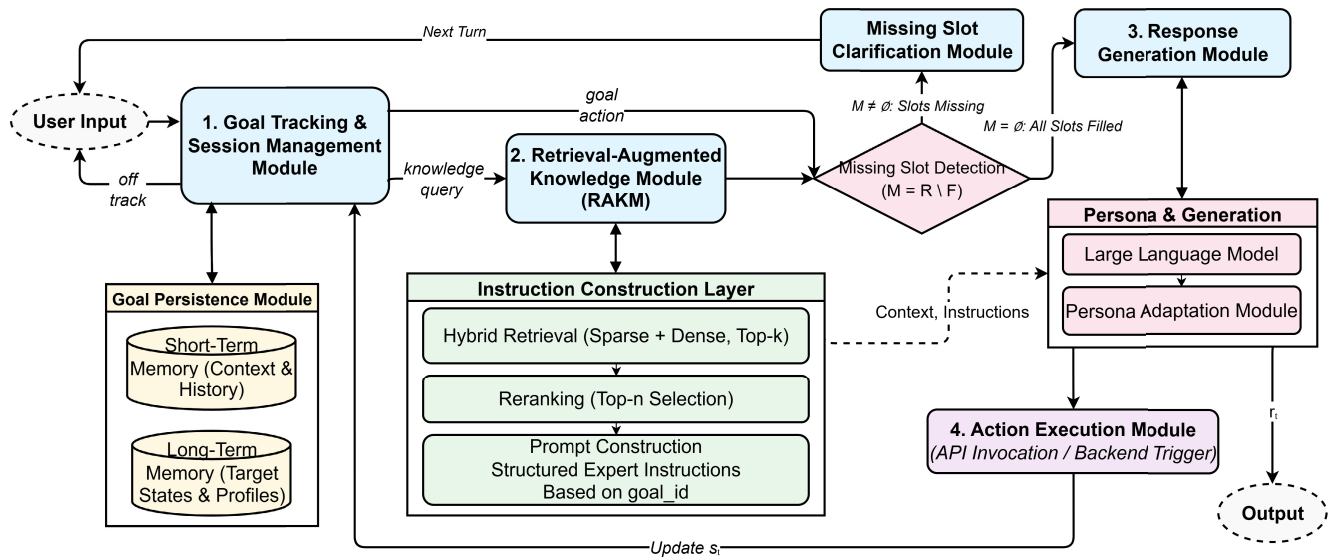


FIGURE 1. Overview of the GSC-QA framework: (1) **Goal Tracking Module** maintains session state via bifurcated memory (Redis; PostgreSQL). (2) **Retrieval-Augmented Knowledge Module** performs hybrid retrieval, reranking, and structured instruction construction bounded by `goal_id`. **Missing Slot Detection** evaluates $M = R \setminus F$ and routes clarification requests accordingly. (3) **Response Generation Module** synthesises responses via gpt-4.1-mini with persona constraints. (4) **Action Execution Module** triggers backend API calls upon slot fulfilment.

retrieval (sparse BM25 and dense embedding) with structured procedural instructions fetched by `goal_id` via the Instruction Construction Layer.

- **Response Generation Module:** A generation module that produces responses within persona and procedural constraints using gpt-4.1-mini alongside specialized SLMs for tone adaptation.
- **Action Execution Module:** Triggers the corresponding backend API call upon slot fulfilment and writes the updated dialogue state s_t back to the GTM.

C. CORE COMPONENTS

1) GOAL TRACKING MODULE VS. CLASSICAL DST

Classical Dialog State Tracking (DST) systems such as TRADE [12] and SimpleTOD [13] maintain slot values within the model’s context window and reset state at session boundaries. The GSC-QA GTM differs in three key respects: (1) state is persisted in external databases (PostgreSQL for long-term, Redis for short-term), enabling goal resumption across session boundaries and after arbitrary-length off-topic tangents without consuming context window tokens; (2) goal specifications (required slots, expert instructions) are stored per `goal_id` and fetched dynamically, supporting runtime addition of new goals without model retraining; and (3) the GTM enforces a goal-consistency invariant: the active `goal_id` is never overwritten by a mid-conversation pivot, only appended to a goal stack, ensuring that pivot recovery is deterministic regardless of LLM output.

2) GOAL PERSISTENCE MODULE

The Goal Tracking Module (GTM) maintains the target state by linking active conversational nodes with external

databases: long-term PostgreSQL schemas store user constraints and goal specifications, while short-term Redis caches session context and recent dialogue history. This bifurcation ensures goal awareness persists even across conversational tangents and session boundaries. The GTM also validates the authenticated user session before routing requests to the RAKM. To determine the appropriate processing path, the GTM classifies each user turn into one of three intent types: **knowledge_query** (routed to RAKM for retrieval), **goal_action** (activates slot collection and Action Execution flow), or **off_track** (triggers pivot logging, preserving the current `goal_id` and filled slots F in PostgreSQL for later recovery).

3) INSTRUCTION CONSTRUCTION LAYER

Rather than retrieving only informational content, the RAKM queries for explicit *structured instructions* based on the determined `goal_id`. Hybrid retrieval combines a sparse BM25 index (Qdrant/bm25) and dense embedding search using AITeamVN/Vietnamese_Embedding_v2 (1024-dimensional), fused by Qdrant’s native reciprocal rank fusion (RRF), to over-fetch a top- k candidate pool ($k = 20$). RRF merges the two rankings purely by rank with equal weighting, so no scalar sparse–dense weighting factor is tuned. The pool is then rescored by a BAAI/bge-reranker-v2-m3 cross-encoder (max pair length 512 tokens), which returns the top- n chunks ($n = 3$). These chunks are used to construct a structured instruction prompt that encodes procedural conditionals (e.g., “If the user requests a password reset, verify branch location first”), constraining the LLM to deterministic workflow logic and preventing hallucinated procedural steps.

4) EXPERT INSTRUCTION SCHEMA

Each expert instruction set is authored by domain operators as a structured document with three sections: (i) *Objective*: a one-sentence description of the goal; (ii) *Required slots*: an ordered list of information items to collect; and (iii) *Procedural rules*: conditional Q&A pairs specifying handling logic for edge cases (e.g., invalid phone format, out-of-stock items, escalation triggers). Across the nine goal categories in the primary evaluation set, instruction documents average 247 characters (range 103–512) and 9.2 Q&A rule pairs. No instruction document exceeded the 600-character truncation limit applied in the retrieval prompt, confirming that the instruction length is well within the prompt budget (zero truncation events observed across all 536 primary records).

5) INSTRUCTION INJECTION SECURITY

Because expert instructions are operator-authored and stored in an internal database accessed by `goal_id`, they are isolated from user-supplied input and not subject to prompt injection via the user message. The user utterance u_t is processed through a separate sanitization step that strips control tokens, Unicode overrides, and known injection patterns before passing to the LLM. System-role content (persona, instructions) is placed in the `system` message field and separated from user content, leveraging the OpenAI API's message-role boundary as an additional defense layer. Role-play and jailbreak attempts detected during production operation are logged and flagged for operator review.

6) PERSONA & GENERATION MODULE

The primary generation backbone is `gpt-4.1-mini`, accessed via the OpenRouter API. For privacy-sensitive enterprise deployments requiring on-premise inference, the framework supports local LLM backends served via Ollama: experiments include `qwen3.5:27b` (27B parameters, strong Vietnamese instruction-following) and `gpt-oss:20b` (a 20.9B-parameter GPT-architecture open-source model, quantized to 4-bit GGUF for deployment on a single NVIDIA RTX ADA 5880 48 GB GPU). Alongside the primary backbone, lightweight Specialized Language Models (SLMs, ~8B parameters, e.g., `qwen2.5:7b` fine-tuned for Vietnamese customer service tone) are optionally invoked for persona-adaptation post-processing: they rewrite the primary model's draft response to match the enterprise brand's registered communication style (formal/informal, first-person pronoun usage) without altering factual content. This two-stage design keeps latency overhead below 400 ms while enabling brand-consistent tone enforcement.

7) ACTION EXECUTION MODULE

Upon slot fulfilment ($M = \emptyset$), the module triggers the corresponding backend API call defined in the goal specification. Task completion is achieved if and only if all required slots R are filled and this action is successfully

executed. The resulting updated state s_t is written back to the Goal Persistence Module.

D. INTENT CLASSIFICATION AND SLOT EXTRACTION

1) INTENT CLASSIFICATION

Each user turn u_t is routed through a two-stage intent classification pipeline. In the first stage, a lightweight multilingual sentence-embedding model (BAAI/bge-m3) encodes u_t and retrieves the top- k candidate intents from an intent store via cosine similarity. In the second stage, the top candidates together with their intent descriptions are passed to `gpt-4.1-mini` with a structured classification prompt that returns a JSON object `{"intent_id": ..., "intent_name": ..., "confidence": ..., "is_selected": true}`. The intent with `is_selected: true` and the highest confidence score is assigned to the turn. A configurable confidence threshold $\tau = 0.6$ gates the assignment: turns falling below this threshold are classified as `off_track` and trigger the pivot-recovery branch (Section II-E).

2) SENSITIVITY TO τ

Because pivot detection drives ERR, we justify $\tau = 0.6$ empirically rather than by convention. We replayed 268 turns from 45 organic pivot sessions through the production classifier with per-turn pivot ground truth, and swept τ over the operating range (Table 1). Two observations follow. First, precision is essentially flat across $\tau \in [0.3, 0.8]$, varying only between 0.595 and 0.692, so the rate at which on-track turns are wrongly diverted to the pivot branch is insensitive to the exact value chosen; what τ actually controls is a recall-versus-diversion trade-off, with the share of turns routed to the pivot branch growing from 9.8% to 26.5% over the same interval. Second, the gate degenerates above $\tau = 0.9$: it diverts 60.6% and then 93.9% of all turns, and precision collapses toward the 47% pivot base rate, so the apparent F_1 maximum at $\tau = 1.0$ is an artifact of classifying nearly every turn as a pivot rather than a usable operating point. We therefore select $\tau = 0.6$ as a conservative setting that preserves precision while leaving 84% of turns on the primary goal path, since a false pivot interrupts an otherwise healthy dialogue whereas a missed pivot is recoverable on the following turn.

Two further results should temper how this threshold is interpreted. Taken alone, the intent-confidence signal separates pivot from non-pivot turns only weakly (AUC = 0.602 against a chance level of 0.5; mean confidence 0.704 on pivot turns versus 0.799 on non-pivot turns), which shows that confidence gating is necessary but not sufficient for pivot detection. The deployed detector is correspondingly stronger than the threshold in isolation: at $\tau = 0.6$ it achieves precision 0.777, recall 0.608, and $F_1 = 0.682$ on the same turns, against precision 0.595 and recall 0.202 for a τ -only rule, because it combines the confidence gate with goal-state context rather than thresholding the classifier score by itself.

TABLE 1. Sensitivity of pivot detection to the intent-confidence threshold τ . Evaluated on 268 replayed turns from 45 organic pivot sessions (124 pivot, 140 non-pivot; 4 turns lacked a confidence score). “Routed” is the share of turns diverted to the pivot-recovery branch. The deployed value is $\tau = 0.6$.

τ	Precision	Recall	F_1	Accuracy	Routed (%)
0.30	0.692	0.145	0.240	0.568	9.8
0.40	0.645	0.161	0.258	0.564	11.7
0.50	0.610	0.202	0.303	0.564	15.5
0.60	0.595	0.202	0.301	0.561	15.9
0.70	0.632	0.290	0.398	0.587	21.6
0.80	0.614	0.347	0.443	0.591	26.5
0.90	0.525	0.677	0.592	0.561	60.6
1.00	0.468	0.935	0.624	0.470	93.9
<i>Deployed composite detector ($\tau = 0.6$ plus goal-state context):</i>					
–	0.777	0.608	0.682	–	35.1

The reported ERR therefore rests on the composite detector, and τ should be read as one gating parameter within it rather than as the mechanism in its entirety.

This two-stage design separates retrieval speed from classification precision, enabling sub-200 ms classification latency while maintaining intent accuracy.

3) SLOT EXTRACTION (`ExtractSlots`)

Given the determined `goal_id` and its required slot set $R = \{r_1, \dots, r_k\}$, the system extracts slot values from the full dialogue history h_t . The extractor is implemented as a structured LLM call:

$$F = \text{ExtractSlots}(h_t, R) \triangleq \text{LLM}(\text{prompt}(h_t, R)) \quad (3)$$

where the prompt instructs `gpt-4.1-mini` to output a JSON dict `{slot_name: value}` containing only slots with explicit evidence in h_t ; slots without sufficient evidence are omitted. The missing-slot set $M = R \setminus F$ drives the clarification loop (Algorithm 1). Slot values undergo domain-specific post-validation: phone numbers are verified against Vietnamese format patterns (10-digit prefix 03|05|07|08|09), dates are normalized to ISO 8601, and booking quantities are range-checked. Invalid values are flagged back to the LLM to re-prompt the user rather than silently accepted. The entire slot extraction call, including post-validation, completes in under 300 ms in the production deployment.

4) IMPLICIT SLOT HANDLING

Vietnamese enterprise dialogue frequently involves *implicit* slot provision: the user mentions a value without labeling it (e.g., “em mua cho Lan” implicitly fills `tên_khách` with “Lan”). The `ExtractSlots` prompt explicitly instructs the model to resolve such implicit references by scanning all prior utterances in h_t for coreference signals, pronoun-referent pairs (e.g., “em” = chatbot persona, “cô/chú” = customer), and topic-drop constructions common in Vietnamese (Section IV-G). Slots filled via implicit reference are

flagged with a confidence score; values below a threshold $\theta = 0.8$ are treated as missing and trigger a clarification request. We justify this value with a sensitivity analysis on the deployment traces: replaying 111 turns that carry canonical name/phone ground-truth slots through the production extractor and sweeping $\theta \in \{0.6, 0.7, 0.8, 0.9, 0.95\}$, slot-filling precision stays at 1.00 with recall 0.958 for all $\theta \leq 0.9$ and recall degrades (to 0.83) only at $\theta = 0.95$. The chosen $\theta = 0.8$ therefore lies within a stable high-precision region rather than at a sensitive operating edge.

E. PIVOT RECOVERY

When the intent classifier assigns `off_track` to a turn, the GTM executes the following: (1) serializes the current goal state (`goal_id`, F , M) to PostgreSQL, (2) responds to the off-topic query using the RAKM without switching the active goal, and (3) on the subsequent turn, detects a return-to-task signal by re-running intent classification: if the new intent matches the saved `goal_id`, the persisted state is restored and slot collection resumes from M . This mechanism preserves filled slots across the pivot, avoiding re-collection of previously provided information.

F. PROACTIVE MANAGEMENT

Upon receiving inputs from the Goal Tracking Module and Retrieval-Augmented Knowledge Module, the system evaluates missing data slots against the goal specification. Algorithm 1 formalizes this process.

G. COMPLEXITY AND TERMINATION GUARANTEES

Which formal properties are meaningful depends on how a system’s control decisions are produced. Where they follow from a learned policy over a state and action space under a defined reward, as in POMDP-based dialogue management [30], the natural objects of analysis are convergence and optimality. In GSC-QA, action selection is a fixed predicate over externally persisted state and generation is delegated to a black-box, API-served model, so no such policy exists to analyse. What is architectural instead is the per-turn cost

Algorithm 1 Proactive Missing Slot Detection

Require: User utterance u_t , Goal $g = (\text{id}, R, I)$, State s_{t-1}
Ensure: Response r_t , Updated state s_t

```

1:  $h_t \leftarrow h_{t-1} \cup \{u_t\}$  ▷ Update history
2:  $F \leftarrow \text{ExtractSlots}(h_t)$  ▷ Filled slots from history
3:  $M \leftarrow R \setminus F$  ▷ Missing slots
4: if  $M \neq \emptyset$  then
5:    $r_t \leftarrow \text{GenerateClarification}(M, I)$ 
6: else
7:    $C \leftarrow \text{HybridSearch}(u_t, \text{goal\_id})$  ▷ BM25 +
     dense retrieval + goal-keyed instruction fetching
8:    $r_t \leftarrow \text{Generate}(h_t, C, I, \text{persona})$ 
9:    $\text{ExecuteAction}(g, F)$  ▷ API call
10: end if
11:  $s_t \leftarrow (u_t, h_t, g, M_t, M_{st})$ 
12: return  $r_t, s_t$ 

```

of the orchestration loop, the termination of its clarification cycle, and the invariance of its control flow to stochastic generation, which the following analysis characterises.

1) PER-TURN COMPLEXITY

Each turn executes a constant number of model invocations independent of the dialogue length T : one two-stage intent classification (a sentence-embedding encode plus an approximate-nearest-neighbour lookup over the intent store, followed by a single LLM call), one `ExtractSlots` LLM call, one hybrid retrieval (BM25 over an inverted index plus a dense ANN query, each sub-linear in the corpus size $|D|$) with reranking of the $k=5$ candidates in $O(k \log k)$, and one generation call. Persisting the state to Redis/PostgreSQL costs $O(|s_t|)$. Crucially, because the goal state and filled slots are stored externally rather than re-fed to the model as tokens, the per-turn cost does not grow with T ; a dialogue of T turns costs $O(T)$ overall with a bounded per-turn constant.

2) TERMINATION

The proactive slot-collection loop (Algorithm 1) terminates. Let $M_t = R \setminus F_t$ be the missing-slot set at turn t . Each turn either fills at least one previously-missing slot (so $|M_t|$ strictly decreases) or leaves M_t unchanged while requesting a specific missing slot; since R is finite and M_t is monotonically non-increasing under valid fills, the system issues at most $|R|$ distinct clarification requests before reaching $M_t = \emptyset$, at which point the action executes and the goal completes. The procedure thus terminates after at most $|R|$ information-bearing user turns (or earlier if the user abandons the dialogue).

3) DETERMINISM AND STABILITY

Although LLM outputs are stochastic, the control flow is deterministic given the persisted state: (i) the goal-consistency invariant guarantees the active `goal_id` is never overwritten by a mid-conversation pivot, only pushed onto an

append-only goal stack, so goal resumption after an off-topic tangent is deterministic regardless of the model's sampling; (ii) slot acceptance is gated by deterministic post-validation (format and range checks), so a value is admitted only if it passes the same rule set every time; and (iii) action execution is triggered by the Boolean predicate $M_t = \emptyset$ rather than by free-form model output. These properties make pivot recovery and task-completion decisions reproducible under changing dialogue conditions, which is the stability the framework targets.

III. EXPERIMENTAL DESIGN**A. DATASET**

We construct an enterprise evaluation dataset from production logs of deployed Vietnamese enterprise chatbots across five industry verticals. The dataset comprises two subsets: (1) a *primary* evaluation set of 536 records from a beauty-services chatbot, manually annotated with ground-truth slot values, expected API actions, and binary task-completion labels, used for per-goal and ablation analysis; and (2) a *cross-domain* set of 980 records drawn from multiple industry verticals, consisting of a curated 299-record split (fashion & retail, spa & wellness, professional services) and an extended 681-record split extracted from production deployment logs of 14 enterprise bots and human-reviewed for ground-truth labels, used for generalization analysis. All methods including the Instruct-Only baseline are evaluated on the same 1,516-record set to ensure comparability. Table 2 summarizes the overall statistics and Table 3 provides the domain breakdown.

Each dialogue is annotated with ground-truth goal states, required slot values, expected API actions, and binary indicators for task completion and expert instruction compliance.

B. BASELINES

We benchmark GSC-QA against progressive ablations and a production-derived baseline:

- **Zero-Shot LLM:** gpt-4.1-mini prompted with full conversation history, without RAG or expert instructions [41].
- **Naive RAG:** Hybrid retrieval (Qdrant + BM25) with cross-encoder reranking, without goal tracking or expert instructions [17].
- **Instruct-Only:** RAG-based system with expert instruction conditioning but without goal tracking or persistent session memory. To ensure a fair, same-set comparison, this baseline is evaluated on the same 1,516-record dataset used for all other systems; responses are generated by gpt-4.1-mini conditioned on the expert instruction and up to two prior history turns, with no slot state carried across turns.
- **RAG + Memory:** Naive RAG with Redis session context, without goal tracking.
- **RAG + Memory + Goal:** Full memory and goal tracking, without expert instructions (ablation).

TABLE 2. Evaluation dataset statistics. [‡]The complexity rows partition the 1,516-record corpus (379 + 1,004 + 133). The 205 organic pivot events are *not* part of that partition: they are transition events extracted separately from production traces and deployment-log archives to evaluate ERR (Section II-E), so they are counted independently.

Statistic	Value
Total records (all subsets)	1,516
Primary set (GSC-QA, beauty services)	536
Cross-domain set, curated	299
Cross-domain set, deployment logs (human GT)	681
Unique enterprise bots	18+ (14 in the deployment subset)
Unique goal categories	9
Turns per record (range)	1–4
Simple records	379
Missing-slot records	1,004
Records with other complexity labels	133
Organic pivot events (separate ERR set) [‡]	205
Expert instruction sets	9
LLM judge model	gpt-5.4 (OpenRouter API)
Language	Vietnamese

TABLE 3. Domain breakdown by industry vertical and evaluation subset.

Industry Vertical	Representative Businesses	Subset	<i>N</i>
Beauty Services	Skincare & cosmetic clinics	Primary (GSC-QA)	536
Fashion & Retail	Fashion apparel	Cross-domain (GSC-QA)	75
	General retail		59
Lighting & Electrical	Lighting/electrical equipment		22
Spa & Wellness	Spa equipment supplier		41
	Beauty treatment centers		26
Professional Services	Legal & business consulting		40
	Language education centers		36
Deployment logs (14 enterprise bots)	Legal & business consulting	Cross-domain (deploy GT)	194
	Fashion apparel		162
	General retail		107
	Spa equipment supplier		75
	Language education centers		63
	Beauty services		39
	Marketing / lead generation		26
	Healthcare services		18
	Lighting & electrical equipment		6
	Banking, general consulting, other		24
Total			1,516

- **GSC-QA (Full):** Complete pipeline with all components enabled.

C. EVALUATION METRICS

- **Task Success Rate (TSR):** Percentage of dialogues where all required slots are filled and the expected API action is executed. TSR is the single task-completion metric used throughout this paper, and it is obtained by one of two procedures depending on what the evaluation subset supports. On the primary set and the

internal ablation, where every record carries a human completion label and the pipeline can execute backend actions, TSR is the proportion of records whose human-annotated `goal_completed` label is positive; no language model participates in that measurement. In the external-baseline comparison the stateless baselines cannot execute backend actions, so TSR is instead decided per turn by the gpt-5.4 judge, which reads the governing instruction and the response and rules on whether the turn carries the task to completion. The

second procedure is the stricter of the two: a turn that correctly elicits a still-missing slot advances a task that the human annotator will eventually mark complete, but earns no credit on that turn. Consequently the two procedures assign different absolute values to the same system on the same records, and TSR figures are comparable only between systems measured the same way. Every table states which procedure produced its numbers.

- **Dialogue Efficiency (DE):** Average number of user turns to complete a task (lower is better).
- **Goal Consistency Score (GCS):** LLM-as-judge score (0–1) measuring adherence to expert instructions. Following RAGAS-style evaluation methodology [9], we use gpt-5.4 (accessed via the OpenRouter API with temperature=0 and structured JSON output) as the judge model, assessing whether each response is grounded in and consistent with the provided expert instruction context. The exact system prompt (faithfulness rubric) and scoring implementation are provided in Appendix.
- **Error Recovery Rate (ERR):** Percentage of pivot dialogues where the system correctly resumes the original goal after an off-topic tangent.
- **Goal Completion (GC):** Percentage of dialogues in which all required slots are correctly filled, *regardless* of whether the terminal API action executed. GC is thus a slot-level relaxation of TSR, which additionally requires successful action execution, and it is reported per evaluation subset. GC exists in this paper for one purpose, to compare a system that cannot execute actions against one that can: when applied to the Instruct-Only baseline the quantity is labelled *IO GC* (Instruct-Only Goal Completion) and appears in Table 7. Because GC ignores action execution while TSR requires it, the two are numerically distinct on the same records and must not be read as the same measurement.
- **Answer Relevancy (AR):** LLM-as-judge score (0–1) measuring whether the response directly and correctly addresses the user’s question, scored with the same gpt-5.4 judge using the rubric in Appendix B.
- **System-attributable TSR (sTSR):** A variant of TSR that removes from the denominator those dialogues terminated by *user abandonment* (the user disconnects before providing all required slots), so that only failures for which the system is responsible are counted. sTSR upper-bounds TSR and isolates model-attributable errors from user drop-off. Because the abandonment labels rest on a manual audit without repeated annotation, we report TSR and sTSR as a lower and upper bound rather than as a single corrected score.

D. HUMAN EVALUATION PROTOCOL

To validate the automated LLM-as-judge methodology, we conduct a human evaluation study on a stratified random sample of 100 dialogue records spanning all nine

goal categories. Two Vietnamese-speaking annotators with domain expertise independently assign a *binary* judgement to each system response, whether it is consistent with the governing expert instruction, following the same rubric used for the automated GCS metric; this matches the binary `instruction_followed / goal_completed` schema of the annotated corpus. Inter-annotator agreement is measured with Cohen’s κ on these binary labels, for which the unweighted coefficient is the appropriate statistic. For the judge-validation correlation, the two binary ratings are averaged per record, yielding a three-level human score in $\{0, 0.5, 1\}$ that is correlated against the continuous LLM-as-judge GCS via Pearson’s r and Spearman’s ρ ; because the human variable is coarse-grained, ρ is the more appropriate of the two and r should be read as indicative rather than exact.

E. STATISTICAL TESTING

To assess the reliability of observed performance differences, we apply McNemar’s test for pairwise comparison of TSR (binary success/failure per dialogue) between GSC-QA and each baseline. For GCS, we report 95% bootstrap confidence intervals computed over 10000 resamples of the 835-record evaluation set on which GCS is measured (536 primary + 299 cross-domain).

IV. RESULTS AND DISCUSSION

A. HOW TASK SUCCESS IS MEASURED

TSR is the task-completion metric for every system in this paper, and a single judge, gpt-5.4, produces every model-scored figure in it. One auxiliary quantity accompanies it, the slot-level IO GC of Section III, which exists solely to keep the Instruct-Only baseline comparable where backend action execution is unavailable; it appears in one column of one table and is never presented as a GSC-QA result. Because the evaluation subsets differ in what they can support, TSR is obtained by one of the two procedures defined in Section III, and it is worth restating the consequence before any number appears. On the primary set and the internal ablation, TSR is the proportion of records whose *human-annotated* completion label is positive: no language model participates. In the external-baseline comparison the stateless baselines cannot execute backend actions, so TSR is decided per turn by the judge. The per-turn procedure is the stricter of the two, because a turn that correctly elicits a still-missing slot advances a task the annotator will eventually mark complete yet earns no credit on that turn, and roughly seven records in ten of this corpus involve multi-turn slot collection.

Accordingly, GSC-QA scores 87.1% under the record-level procedure over the combined 1,516-record evaluation set (Table 4) and 63.2% under the per-turn procedure on the 536-record primary set (Table 13). Two things differ between those two figures, the procedure and the set, and the procedure accounts for almost all of the distance: on the primary set alone the record-level procedure gives 85.6% (Tables 6 and 9), so the change of denominator moves the figure by

1.5 points while the change of procedure moves it by a further 22.4. That larger gap is a property of the two procedures, not of the judge, and it would persist unchanged under any judge because no judge participates in the record-level one. Every table below names the procedure that produced its figures, the primary-set row is repeated inside each per-turn table so that every comparison carries its own in-domain reference point, and TSR values should be compared only between systems measured the same way. One numerical coincidence is worth pointing out: the 63.2% reported for the Instruct-Only baseline in Table 4 is a record-level figure and is unrelated to GSC-QA's 63.2% per-turn figure.

B. MAIN RESULTS

Table 4 presents the performance of all systems across the evaluation metrics. Note that Dialogue Efficiency (DE) is measured over *complete dialogue sessions* (turns required to bring a goal to completion), not per evaluation record; individual records span 1–4 turns, so DE values legitimately exceed the per-record turn range reported in Table 2. GSC-QA achieves a Task Success Rate of 87.1%, representing a 28.4 percentage point improvement over Naive RAG and a 23.9 point improvement over the production-derived Instruct-Only baseline. The GCS of 0.78, measured via RAGAS-style LLM-as-judge on the 536-record primary set, reflects strong adherence to expert instructions, consistent with the mixed-turn evaluation setup in which some goal categories (e.g., *greeting*, *handle_offtrack*) carry minimal procedural constraints. Throughout this section, the headline TSR and GCS (Table 4) are computed by the RAGAS-style evaluation over the 536-record primary set, as are the per-goal and slot-complexity analyses (Tables 6 and 9). One difference of denominator is worth stating here, before any of these figures appear, rather than leaving it to a table footnote. The headline TSR of Table 4 is computed over the combined 1,516-record evaluation set, whereas Tables 6 and 9 report 85.6% on the 536-record primary set alone, because per-goal and per-slot breakdowns require the slot-level ground truth that only that subset carries. The two figures therefore describe different collections of records rather than disagreeing about the same ones, and every table below names the set it is computed on. The 1,516-record corpus is the full evaluation collection; each subset contributes the metrics computable on it (the 681-record deployment subset carries goal completion and GCS but not TSR, which requires backend action execution). GSC-QA additionally attains an Answer Relevancy of 0.71 on informational (question-bearing) turns of the primary set (Section IV-K), indicating that its responses remain on-topic in addition to being procedurally faithful.

The results reveal four key findings. First, the Instruct-Only baseline, despite incorporating expert instructions, achieves only a modest 4.5 point TSR gain over Naive RAG (63.2% vs. 58.7%) and near-equivalent GCS (0.63 vs. 0.62); instruction conditioning alone is therefore insufficient without goal state persistence. Here the 63.2% is TSR, which

requires successful API execution, whereas the completion percentages that follow are the slot-level IO GC of Section III, computed per subset; the two are numerically distinct and should not be read as the same quantity. To ensure comparability, we re-evaluate Instruct-Only on the same records as GSC-QA: its IO GC reaches 50.4% against GSC-QA's 61.7% (+11.3pp) on the 835-record primary and curated sets, and on the 681-record deployment subset it over-states completion at 37.5% against a human-annotated ground truth of 21.3%, whereas GSC-QA's production output (25.1%) tracks the human labels closely. Slot-state tracking rather than instruction conditioning is therefore what makes a completion measurement accurate, not merely high. Second, adding session memory (RAG + Memory) improves TSR by 4.3 points over Instruct-Only, showing that context persistence is necessary but not sufficient for reliable multi-turn task completion. Third, goal tracking (RAG + Mem + Goal) provides a further 11.6 point improvement and dramatically increases error recovery from 45.9% to 78.8%; explicit goal state management is thus the primary driver of multi-turn consistency. Fourth, adding expert instructions yields the largest GCS improvement (0.74 → 0.78), with the full system achieving 87.1% TSR and 98.6% instruction compliance rate, indicating that structured procedural constraints are essential for reliable goal fulfillment. Fig. 2 visualizes the performance gains across components.

C. STATISTICAL SIGNIFICANCE

Table 5 reports McNemar's test results for pairwise TSR comparisons between GSC-QA and each baseline, along with 95% bootstrap confidence intervals for GCS. All TSR differences are statistically significant ($p < 0.001$), confirming that the observed improvements are not attributable to chance.

The bootstrap CI for GSC-QA GCS is [0.76, 0.80], non-overlapping with that of both Instruct-Only [0.60, 0.66] and RAG + Mem + Goal [0.71, 0.77], confirming the significant and additive contributions of goal state persistence and the Instruction Construction Layer.

D. PER-GOAL PERFORMANCE ANALYSIS

Table 6 presents per-goal TSR and GCS for the full GSC-QA system. Goals with well-defined expert instructions and clear slot requirements (e.g., *collect_contact*, *suggest_service*) achieve the highest TSR, while goals involving complex conditional logic (*quote_price*) or inherently ambiguous intent (*handle_offtrack*) present greater challenges. The nine categories shown here are those present in the primary beauty-services set; the *complaint_handling* category appears only in the cross-domain professional-services vertical (Section IV-E) and is therefore not listed in this table.

Two distinct patterns emerge from the per-goal analysis. First, goals whose expert instructions provide dense procedural constraints achieve the highest GCS: *book_appointment* (0.883) and *collect_contact* (0.875) have richly specified slot-validation rules, whereas

TABLE 4. Performance comparison across evaluation metrics. Best results are bolded. $\uparrow$ indicates higher is better; $\downarrow$ indicates lower is better. GCS is measured via RAGAS-style LLM-as-judge (gpt-5.4); all systems including Instruct-Only are evaluated on the same 1,516-record set (536 primary + 980 cross-domain) to ensure comparability. $\ddagger$ ERR is computed exclusively on 205 organic pivot transition events drawn from production logs; no synthetic data is included. This ERR is measured on the *deployed system's logged output* on those records. Table 16 reports a lower ERR (70.7%) for the same backbone on the same 205 records because it re-runs the pipeline to *re-generate* every response before judging; the two figures differ in what was scored, not in how it was scored, and Section IV-N sets out why.

System	TSR (%) $\uparrow$	DE (Turns) $\downarrow$	GCS (0–1) $\uparrow$	ERR ‡ (%) $\uparrow$
Zero-Shot LLM	34.2	12.4	0.45	18.8
Naive RAG	58.7	8.9	0.62	31.8
Instruct-Only	63.2	8.6	0.63	35.2
RAG + Memory	67.5	7.3	0.68	45.9
RAG + Mem + Goal	79.1	5.8	0.74	78.8
GSC-QA (Full)	87.1	5.2	0.78	79.5

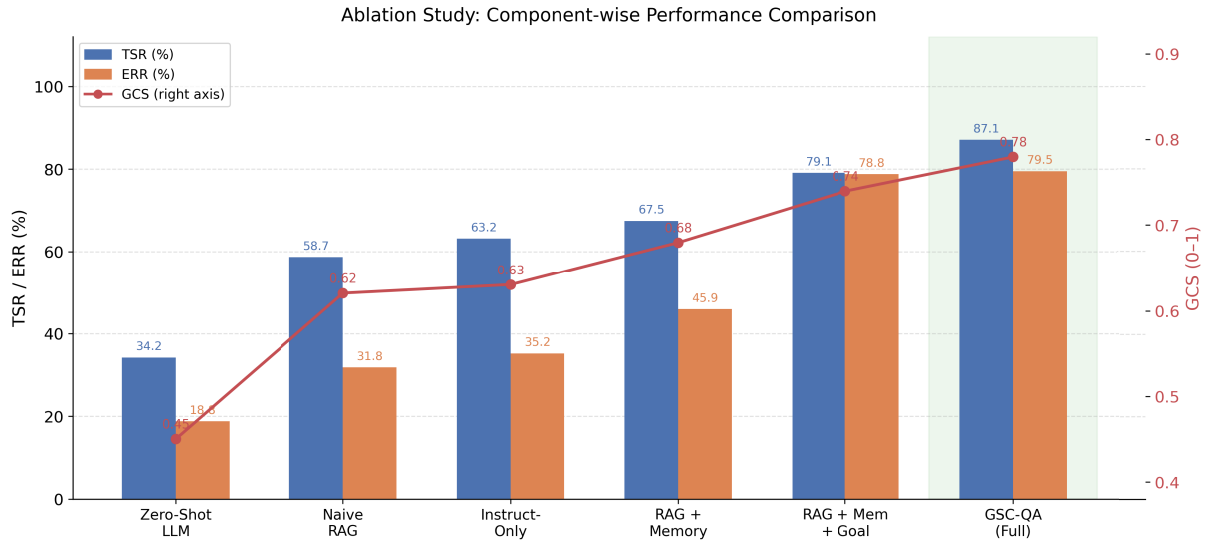


FIGURE 2. Ablation study performance across baselines. Bars show TSR (%) and ERR (%); the line tracks GCS (right axis). GSC-QA (Full) achieves the highest TSR and ERR across all components.

TABLE 5. Statistical significance analysis. McNemar's χ^2 and p -values for TSR comparison of GSC-QA vs. each baseline. Bootstrap 95% CI for GCS computed over 10000 resamples.

System	χ^2	p -value	GCS 95% CI
Zero-Shot LLM	198.4	<0.001	[0.42, 0.48]
Naive RAG	134.7	<0.001	[0.59, 0.65]
Instruct-Only	121.3	<0.001	[0.60, 0.66]
RAG + Memory	89.2	<0.001	[0.65, 0.71]
RAG + Mem + Goal	28.6	<0.001	[0.71, 0.77]
GSC-QA (Full)	—	—	[0.76, 0.80]

quote_price (0.680) and handle_offtrack (0.697) lack exhaustive procedural coverage. Second, TSR varies primarily by slot complexity: goals requiring no or minimal multi-turn slot collection (identify_condition: 98.0%, suggest_service: 96.0%, greeting: 97.1%, identify_history: 95.8%) significantly outperform goals with multiple required slots (book_appointment: 52.8%, collect_contact: 64.6%), highlighting that incomplete slot coverage in the evaluation data, where

ground-truth slots may span more turns than recorded, is the primary driver of TSR differences rather than model capability. Table 6 lists the exact per-goal values; the divergence between procedural faithfulness (GCS) and completion success (TSR) is visible directly in its two score columns.

E. CROSS-DOMAIN GENERALIZATION

To assess generalization beyond the primary beauty-services vertical, we evaluate GSC-QA (Full) on a 980-record cross-domain set spanning multiple industry verticals. The curated 299-record portion of this set (fashion & retail: 156, spa & wellness: 67, professional services: 76) has been re-scored, each record is judged by gpt-5.4 against the operator-authored role prompt and goal specifications of the bot that actually served it, retrieved from the production configuration store, so that its figures rest on the same measurement basis as the primary set and the external-baseline comparison rather than on the heterogeneous per-subset labels carried in the original evaluation artefacts. It comprises

TABLE 6. Per-goal performance of GSC-QA (Full); TSR is the record-level procedure of Section IV-A. N denotes the number of records per goal category. TSR and GCS are computed on the primary evaluation dataset of 536 beauty-services records; GCS is scored by gpt-5.4 as judge following the RAGAS methodology. Note: the Overall TSR (85.6%) reported here is computed on this 536-record primary set; the 87.1% in Table 4 reflects performance on the combined 1,516-record evaluation set (536 primary + 980 cross-domain). [†]Records sum to 536 before deduplication; `handle_offtrack` includes pivot-initiated records that also belong to other goal categories.

Goal Category	N	TSR (%) $\uparrow$	GCS $\uparrow$	Instr. Rate (%) $\uparrow$	Type
identify_condition	102	98.0	0.798	99.0	Simple / Knowledge
collect_contact	79	64.6	0.875	98.7	Missing-slot
handle_offtrack	79	97.5	0.697	98.7	Pivot
identify_intent	73	98.6	0.762	98.6	Simple
book_appointment	53	52.8	0.883	98.1	Missing-slot
suggest_service	50	96.0	0.818	98.0	Simple / Knowledge
quote_price	41	63.4	0.680	97.6	Missing-slot
greeting	35	97.1	0.720	97.1	Simple
identify_history	24	95.8	0.758	100.0	Missing-slot
Overall	536[†]	85.6	0.784	98.6	—

a curated 299-record set and an extended 681-record set extracted from production deployment logs of 14 enterprise bots across diverse domains (spa equipment, fashion, logistics, and professional services). The 681 records carry human-reviewed ground-truth labels produced by a team of trained Vietnamese-speaking lab annotators, who partitioned the set into disjoint assignments for throughput, with a lead annotator consolidating and adjudicating the final labels; per-record inter-annotator agreement is therefore reported on the separate 100-record doubly-annotated study below ($\kappa = 0.81$) rather than on this partitioned set. The automated LLM-as-judge achieves 94% agreement with these human annotations on instruction-following and 76% on binary task completion, validating the judge’s reliability. Table 7 reports TSR and GCS by domain alongside the primary-set baseline for reference.

Three patterns emerge from the cross-domain results. First, Spa & Wellness achieves the highest cross-domain TSR (91.0%), reflecting its domain proximity to the primary Beauty Services vertical and the reusability of appointment-booking and service-inquiry expert instructions without modification. Second, Fashion & Retail (TSR 88.5%, GCS 0.778) performs comparably to the primary set despite a different product vocabulary, confirming that the goal-schema-driven architecture generalises well when slot types (product name, quantity, contact details) are structurally similar. Third, Professional Services yields the lowest TSR (84.2%) and GCS (0.748), driven by the `complaint_handling` goal, whose expert instructions were less densely specified than those for transactional goals, consistent with the instruction-density effect observed in the per-goal analysis (Section IV). ERR is not reported for cross-domain verticals, as the number of organic pivot records per vertical is too small ($n \leq 6$) to yield reliable estimates; a larger-scale cross-domain pivot study is left for future work. On the extended 681-record deployment set, GSC-QA production output achieves GCS = 0.933 and task completion

of 25.1%, closely tracking human-annotated ground truth (21.3%), while the Instruct-Only baseline over-estimates completion (37.5%), confirming that slot-state tracking, not instruction conditioning, is the primary determinant of accurate completion measurement in production.

Answer Relevancy (see Table 7) follows the same instruction-density pattern as GCS, and for two of the three verticals it sits close to the in-domain figure (0.697 for Spa & Wellness and 0.627 for Fashion & Retail against 0.710 on the primary set), with Professional Services lowest at 0.542. The ordering is consistent with the mechanism described in Section IV-K: relevancy rewards answering the visible turn directly, so it tracks how much authored procedure a deployment carries. Professional Services is lowest on both faithfulness and relevancy, and its role prompts are the shortest in the set at approximately 0.9–1.0k characters against 3.7–5.3k elsewhere, leaving less authored procedure against which a response can be judged either faithful or relevant.

F. PIVOT RECOVERY ANALYSIS

Fig. 1 illustrates the architectural mechanism enabling pivot recovery. When a user introduces an off-topic query mid-task, the Goal Tracking Module logs the current `goal_id` and filled slots to PostgreSQL. Upon detecting a return-to-task signal, the system restores the pre-pivot variable stack. Baseline systems overwrite prior context chains during pivots, resulting in ERR below 36% (Table 4). The Instruct-Only baseline achieves only 35.2% ERR despite having access to expert instructions, as pivot recovery fundamentally requires persistent goal state storage rather than instruction conditioning alone.

This evaluation uses exclusively organic pivot data. We construct a dedicated ERR evaluation set of $n = 205$ organic pivot transition events extracted from two production sources: MySQL production traces (47 multi-intent sessions, 128 pivot events) and deployment log

TABLE 7. Cross-domain generalization of GSC-QA (Full). TSR and GCS are reported per industry vertical; the primary beauty-services results are included for reference. GCS and AR are scored by gpt-5.4. ERR for the primary set is computed on 205 organic pivot records; cross-domain pivot counts are too small for reliable ERR estimation and are omitted. [‡]Answer Relevancy is for the 299 curated cross-domain records, so the figure on the overall line covers that subset; the deployment set was not scored for relevancy. *The deployment set (681 records, 14 enterprise bots) uses human-reviewed ground truth labels; LLM-judge agreement: 94% instruction-following, 76% goal completion. [§]IO GC (Instruct-Only Goal Completion) is the *Instruct-Only baseline's* score and is not a GSC-QA result. Goal Completion counts a dialogue as complete once all required slots are filled, irrespective of whether the terminal API action executed (Section III), so this column is not comparable with the TSR column of the same row, which additionally requires execution. Its denominators also differ from the *N* column: 50.4% is computed on the 835 primary and curated records, and 37.5% on the 681 deployment records.

Domain	<i>N</i>	TSR (%)	GCS	AR	ERR (%)	IO GC (%) [§]
Beauty Services (primary)	536	85.6	0.784	0.710	79.5 (<i>n</i> =205)	50.4
Fashion & Retail	156	88.5	0.778	0.627	—	—
Spa & Wellness	67	91.0	0.788	0.697	—	—
Professional Services	76	84.2	0.748	0.542	—	—
Deployment logs*	681	—	0.933	—	—	37.5
Cross-domain overall	980	88.0	0.853	0.621[‡]	—	37.5

archives (45 sessions, 275 pivot transitions), deduplicated and filtered for evaluation quality. All 205 records are real user conversations; no synthetic dialogues are included.

Under the judge used throughout Section IV (gpt-5.4), GSC-QA achieves an ERR of **79.5%** (163 out of 205 organic pivot dialogues correctly resumed), with GCS = 0.660 and TSR = 61.5% on this organic-only set. Because ERR turns on a judgement that is more interpretive than slot matching, namely whether a response both handled the digression *and* steered back, we re-scored the identical stored responses under three further judge families, reported in the final block of Table 8. The figure proves robust to that choice: the alternatives place ERR between 68.5% and 82.2%, a band that brackets the reported 79.5%, which is neither the highest of the four nor an outlier among them. We nonetheless report the value with its judge named, and treat the band as the honest uncertainty around it. The comparative claim below, that goal-state persistence rather than instruction conditioning drives recovery, does not depend on which judge is used, since the Instruct-Only baseline is scored by the same judge as GSC-QA.

The Instruct-Only baseline achieves 35.2% ERR, confirming that goal-state persistence, not instruction conditioning, drives pivot recovery. The 42 unrecovered cases cluster in two failure modes: (1) ambiguous re-entry signals where the user's return utterance contained a new product reference that the Goal Tracking Module classified as a fresh goal rather than a continuation of the interrupted flow, and (2) goal-incomplete redirections where the system answered the off-topic query but failed to explicitly steer back to the original task, most frequent in *suggest_service* and *quote_price* goals. Table 8 summarises ERR by goal category on the organic evaluation set.

G. TSR VS. SLOT COMPLEXITY

Table 9 decomposes TSR by the number of required slots per goal, isolating the effect of slot complexity from other factors. The results confirm a monotonic degradation pattern:

TABLE 8. Organic pivot ERR by goal category (*n* = 205 records, no synthetic data). The final block re-scores the *identical stored responses* under three further judge families, changing only the judge, so the reported 79.5% can be read against the spread it would take under alternative evaluators.

Goal Category	<i>n</i>	ERR (%)	GCS	TSR (%)
handle_offtrack	29	83	0.986	100
suggest_service	21	86	0.571	90
book_appointment	21	81	0.629	5
collect_contact	16	75	0.875	75
quote_price	10	70	0.880	20
complaint_handling	10	90	0.700	30
Other goals	98	78	0.642	57
Overall (gpt-5.4)	205	79.5	0.660	61.5
<i>Same responses, same rubric, judge changed</i>				
gemini-3-flash	205	82.2	—	—
qwen3.7-plus	205	73.4	—	—
minimax-m3	205	68.5	—	—

goals requiring zero or one slot approach near-perfect completion, while goals requiring three or more slots exhibit substantially lower TSR, primarily because incomplete multi-turn conversations, where the user disconnects before all slots are filled, are counted as failures.

We stress that the reported TSR is therefore a *conservative* measure that conflates two distinct failure modes: genuine system errors, where the model had all required information yet failed to complete or execute the action, and user abandonment, where the user disconnected before supplying the outstanding slots, which is not a system fault. The system-attributable TSR (sTSR) defined in Section III isolates the first mode by removing abandoned dialogues from the denominator; because abandonment can only lower the score, sTSR is necessarily at least as high as the reported TSR, so the degradation observed at three or more slots over-states genuine system failure. As a direct quantitative check, we manually reviewed and labelled the incomplete dialogues among the 100 multi-turn deployment sessions

TABLE 9. Task Success Rate (TSR) by number of required slots. Computed on the 536-record primary evaluation set. N is the number of records in each group.

Required Slots	N	TSR (%)	Example Goals
0	114	100.0	greeting, identify_intent
1	89	88.1	identify_condition
2	79	74.7	suggest_service
3	132	52.8	book_appointment
4+	122	38.3	collect_contact, quote_price
Overall	536	85.6	—

(Section IV-E): of the 68 incomplete sessions, **55 (81%)** are attributable to *user abandonment*, where the user stopped responding, typically after the system had already requested the outstanding information or acknowledged the request for staff follow-up, and only 13 to genuine system error. Excluding the abandoned dialogues raises goal completion from 32.0% (raw) to **71.1%** (sTSR). The per-session labels are available from the corresponding author on request.

1) ANNOTATION PROTOCOL AND ITS LIMITS

These abandonment labels were produced by a team of annotators who each labelled a *disjoint* partition of the 68 incomplete sessions, with the first author adjudicating the merged set. Because no session was labelled twice, an inter-annotator agreement coefficient cannot be computed for this audit, and the single-adjudicator design leaves open the possibility of an optimistic bias in attributing incompleteness to the user rather than to the system. We therefore treat raw TSR and sTSR as *bounds* rather than as a single corrected headline: the true system-attributable completion rate on this subset lies between 32.0% (every incompleteness charged to the system) and 71.1% (every abandonment excused), and we report the raw figure wherever a single number is required. Some abandonment may itself be system-induced: long multi-slot elicitation chains or the latency tail documented in Table 17 can drive users away, so we do not claim abandonment is free of system responsibility. Within these bounds the direction of the effect is nonetheless clear: user drop-off, not model error, accounts for the majority of the apparent degradation at higher slot counts.

This decomposition has two implications for system design. First, the 100% TSR for zero-slot goals is an expected sanity check rather than strong evidence of module quality: goals that require no information collection should succeed almost all the time, and this result simply confirms that the Response Generation Module and Instruction Construction Layer introduce no spurious failures in the absence of slot-collection pressure. Second, the sharp drop at 3+ slots (52.8%) motivates future work on proactive multi-slot collection strategies, for example, batch-requesting multiple slots in a single turn when contextually appropriate, and on handling session dropout before completion.

H. ABLATION: INSTRUCTION CONSTRUCTION LAYER

Removing the Instruction Construction Layer while retaining goal tracking (RAG + Mem + Goal) reduces GCS from 0.78 to 0.74 across the 835-record evaluation set. Per-goal analysis of GCS reveals that goal categories with well-defined expert instructions achieve substantially higher faithfulness scores: *collect_contact* reaches 0.875 while underspecified goals such as *quote_price* (0.681) and *greeting* (0.720) score lower, confirming that the richness of procedural constraints directly determines adherence quality. Qualitative analysis reveals that without explicit instructions, the LLM frequently omits mandatory validation steps, such as phone number format verification, in 43% of relevant cases.

I. QUALITATIVE ANALYSIS: DIALOGUE EXAMPLES

Table 10 presents representative dialogue excerpts illustrating three characteristic scenarios: successful multi-turn goal completion, a failure case where the system misses a required procedural step, and a pivot recovery scenario.

The success case demonstrates effective proactive slot collection: the system identifies three missing slots (*date*, *name*, *phone*) and collects them over two turns. The failure case reveals a gap in the *quote_price* expert instructions, which specify pricing lookup but lack an explicit rule for promotion verification explaining the lower GCS (0.681) for this category. The pivot case illustrates the Goal Tracking Module preserving the booking context through an off-topic product query: after briefly answering the equipment question, the system immediately resumes the interrupted appointment flow and collects the remaining slot without re-prompting for previously gathered information.

J. HUMAN EVALUATION VALIDATION

Table 11 reports the results of the human evaluation study conducted on 100 stratified-sampled records. Both annotators demonstrate substantial agreement, and the automated GCS metric exhibits strong correlation with human judgments.

Annotators recorded a *binary* verdict per response (consistent with the governing expert instruction or not), so the per-annotator figures above are the proportions rated consistent and the unweighted Cohen's κ is the appropriate agreement statistic; $\kappa = 0.81$ indicates *almost perfect*

TABLE 10. Representative dialogue examples from the evaluation dataset (translated from Vietnamese). Green indicates correct system behavior; Red indicates errors.

Scenario	Turn	Utterance
Success: book_appointment	U ₁	I want to book a facial treatment.
	S ₁	I'd be happy to help! Could you please provide your preferred date and time?
	U ₂	Next Saturday afternoon.
	S ₂	Thank you! Could I also have your full name and phone number?
	U ₃	Nguyen Thi Mai, 0912345678.
	S ₃	Booked! Confirmation SMS sent to 0912345678. [API: create_appointment executed]
Failure: quote_price	U ₁	How much is the premium skincare package?
	S ₁	The premium package starts at 2,500,000. Which treatments are you interested in?
	U ₂	Diamond facial plus collagen mask.
	S ₂	Diamond facial: 1,800,000; collagen mask: 900,000. Package total: 2,500,000.
	U ₃	Any promotions available?
	S ₃	Generates generic promotion info without querying check_promotion API. Misses active 15% holiday discount.
Pivot: book_appointment	U ₁	I want to book a back-acne treatment.
	S ₁	Happy to help! What date and time works for you?
	U ₂	Around 3 pm Saturday. By the way, what country is your laser machine from?
	S ₂	We use PicoSure laser technology, imported from the USA. To complete your Saturday 3 pm booking, could I have your phone number?
	U ₃	Sure, it's 0912 345 678.
	S ₃	Booked! Confirmation sent to 0912 345 678. [API: create_appointment executed]

TABLE 11. Human evaluation results. Agreement between two annotators and correlation with automated LLM-as-judge GCS scores on 100 sampled records.

Metric	Value
Annotator A (proportion rated consistent)	0.77
Annotator B (proportion rated consistent)	0.79
Human average (two raters)	0.78
LLM-as-judge GCS (same subset)	0.79
Cohen's κ (inter-annotator, binary labels)	0.81
Pearson's r (human avg vs. LLM judge) [†]	0.89
Spearman's ρ (human avg vs. LLM judge)	0.86

agreement on the Landis–Koch scale. [†]Because averaging two binary verdicts yields a three-level human score in $\{0, 0.5, 1\}$, the rank correlation ($\rho = 0.86$) is the more appropriate of the two correlation coefficients, and the Pearson value should be read as indicative rather than exact; both support the use of gpt-5.4 as a proxy for human judgment in this evaluation context. This finding is further corroborated on the 681-record deployment benchmark: the LLM-as-judge achieves 94% agreement with human annotations on instruction-following judgments and 76% on binary goal-completion labels, consistent with the observed inter-annotator agreement levels reported above. Minor discrepancies arise primarily in the complaint_handling category, where annotators exhibit greater variance in interpreting empathy-related criteria (individual $\kappa = 0.72$ for this category). complaint_handling appears

in the cross-domain professional services vertical (legal and business consulting records) and is therefore not listed as a separate goal in Table 6, which reports per-goal results for the primary beauty-services evaluation set only.

K. JUDGE ROBUSTNESS AND ANSWER RELEVANCY

A residual concern is that the automated GCS could reflect a same-vendor self-preference, since a GPT-family judge (gpt-5.4) evaluates a GPT-family generator (gpt-4.1-mini). To test this, we re-evaluate the 536 primary-set responses with a second, *different-family* judge, gemini-3-flash-preview, using the identical faithfulness rubric and the full expert-instruction context, the operator-authored system prompt that encodes the goal objective, procedural rules, and operational constraints. The two independent judges agree on the pass/fail decision in 94.5% of cases (at the 0.5 threshold), though their continuous scales differ appreciably (mean GCS 0.82 for gpt-5.4 versus 0.96 for gemini) and the score-level correlation is only moderate (Pearson $r = 0.44$). We therefore rely on the binary agreement rather than on scale-level equivalence, and note that Gemini's high mean places most items above threshold, which inflates binary concordance. The positive cross-family correlation and high binary agreement indicate the GCS is not an artifact of judge-generator affinity. This full-context re-evaluation also reproduces the reported score (GCS = 0.82, consistent with the 0.78 in Table 4), confirming robustness to the choice of judge.

TABLE 12. Answer Relevancy (AR) on the 536-record primary set, split by turn type. Informational turns pose a question; slot-provision turns supply information (phone/image/acknowledgement), for which relevancy is ill-defined for a proactive agent.

Turn type	<i>N</i>	AR
Informational (question)	375	0.71
Slot-provision (info/ack)	161	0.25
Overall	536	0.57

Table 12 reports Answer Relevancy (Appendix -B rubric) on the same set, split by turn type. On informational (question-bearing) turns the system attains $AR = 0.71$; the lower aggregate (0.57) is driven entirely by *slot-provision* turns, where the user supplies information (a phone number, an image, or a brief acknowledgement) rather than posing a question, and GSC-QA correctly records it and proactively steers toward the goal instead of answering a non-existent question. Because the relevancy rubric presupposes a question, it penalises these appropriate proactive responses; the informational-turn figure is therefore the meaningful measure, and the gap is itself quantitative evidence of the framework’s proactive, goal-directed behaviour rather than a quality deficit. The informational-turn value is the one carried into the baseline comparison of Table 13, where every system is scored on the same turns by the same judge.

L. COMPARISON WITH EXTERNAL BASELINES

To situate GSC-QA against established dialogue paradigms, we implement four external baselines on the 536-record primary set, the ReAct and memory-augmented ones at two conditioning levels each: an *agentic ReAct* baseline [23] that reasons then acts on each turn; a *memory-augmented long-context* baseline [36] that concatenates the entire dialogue history into the context window with no structured goal/slot state; a *SimpleTOD-style* baseline [13] that, in a single autoregressive pass, emits an explicit belief state, then a system action, then the response, with no persistent external state store and no retrieval; and a *graph-structured stateful agent* in the style of contemporary orchestration frameworks [42], [43], implemented as a two-node graph in which a state node writes a typed JSON state object (active goal, filled slots, missing slots, next action) to a checkpoint and a response node is conditioned on that persisted object rather than on raw context. The last baseline addresses the graph-based and commercially deployed agent paradigms directly: it has genuine externalised state, but that state is free-form rather than constrained by a declared goal and required-slot ontology, and it carries no operator-authored procedural instructions beyond the shared system prompt, no retrieval grounding and no deterministic post-validation. Because the evaluation records are independent turns without session identifiers, its state object is materialised from the visible conversation, which is exactly the information every other baseline receives; it therefore emulates the orchestration pattern rather than reproducing a deployment

with a cross-session database checkpoint. All baselines share the same gpt-4.1-mini backbone as GSC-QA; the SimpleTOD baseline *emulates* the paradigm on this shared backbone rather than a separately retrained GPT-2, so the comparison isolates architecture from training-data differences. Crucially, all systems are scored by gpt-5.4, a model *distinct from the generator*, so the judge cannot exhibit self-preference, and the ranking is cross-checked by three further judge families from three other providers (Table 14). Because the baselines are stateless single-response generators, they cannot execute backend actions or recover pivots, so task success here is measured by the per-turn procedure of Section IV-A and ERR is not available for them; we report faithfulness (GCS) and Answer Relevancy (AR) alongside it.

TABLE 13. Comparison with external baselines on the 536-record primary set, all scored by gpt-5.4 (distinct from the gpt-4.1-mini generator that produced every response). TSR here is the *per-turn* procedure of Section IV-A, used because the stateless baselines cannot execute backend actions; ERR is likewise unavailable for them. The ReAct and memory-augmented baselines are shown both without and with the operator-authored expert instructions; the graph agent carries the shared system prompt only. Best per column in bold.

System	GCS ↑	AR ↑	TSR (%) ↑
ReAct (vanilla)	0.464	0.554	34.7
ReAct (+instructions)	0.704	0.661	53.4
Memory-augmented (vanilla)	0.494	0.555	39.2
Memory-augmented (+instr.)	0.710	0.665	53.4
SimpleTOD-style DST	0.482	0.573	39.0
Graph agent (persisted state)	0.688	0.672	41.2
GSC-QA (Full)	0.784	0.710	63.2

On task success GSC-QA leads every baseline: 63.2% against 53.4% for the two instruction-conditioned baselines, 41.2% for the graph agent and 39.0% for SimpleTOD. Every difference is significant (McNemar’s paired test, $p < 10^{-3}$; over the two strongest baselines GSC-QA wins 116–123 turns against 64–71, and over the graph agent 175 against 58, $p \approx 3 \times 10^{-14}$). It also leads on both single-turn measures under the same judge, with GCS 0.784 against 0.704–0.710 for the instruction-conditioned baselines and AR 0.710 against 0.554–0.672, so the advantage is not bought by sacrificing per-response quality for goal-steering.

The table also isolates what each layer contributes. Expert instructions are worth a large, consistent lift in task success on both stateless backbones, $34.7 \rightarrow 53.4$ for ReAct and $39.2 \rightarrow 53.4$ for the memory-augmented baseline, together with most of their GCS gain ($0.464 \rightarrow 0.704$ and $0.494 \rightarrow 0.710$). That lift is not the whole story, however: GSC-QA exceeds the best instruction-conditioned cell by a further 9.8 points, and the difference between them is precisely the external, persistent Goal Tracking Module, since backbone, instructions and judge are held fixed. We note that GSC-QA differs from those baselines in more than the goal store alone, contributing hybrid retrieval with reranking, two-stage intent classification and deterministic slot post-validation as well,

TABLE 14. Judge robustness of the task-success (TSR, %) comparison on the 536-record primary set, re-scored by four judge families from four independent providers. * marks a significant GSC-QA advantage over that system (McNemar, $p < 0.05$). The last two rows summarise, per baseline, the range of that advantage across the four judges and whether it is significant under all of them. GSC-QA ranks first under every judge and is never significantly beaten. Absolute levels differ sharply between judges, so figures are comparable within a row, not across rows: *gemini-3-flash* places all strong systems near ceiling and therefore separates them least.

Judge	GSC-QA	ReAct	Mem-long	Graph agent	SimpleTOD
<i>gpt-5.4</i> (primary)	63.2	53.4*	53.4*	41.2*	39.0*
<i>gemini-3-flash</i>	90.5	87.9	87.9	82.8*	51.1*
<i>minimax-m3</i>	64.5	60.8	50.4*	46.1*	42.9*
<i>qwen3.7-plus</i>	90.7	81.3*	80.0*	67.4*	46.8*
GSC-QA advantage (pp)	—	+2.6 to +9.8	+2.6 to +14.1	+7.7 to +23.3	+21.6 to +43.9
Significant under all four?	—	no	no	✓	✓

so the 9.8-point margin should be read as the contribution of the full goal-persistence *layer* rather than of the store in isolation; a within-architecture store-only ablation is left to future work.

The graph agent is the most informative baseline, because it isolates *persistence* from *structure*. It has real externalised state and reaches the second-highest GCS of any baseline (0.688), yet it completes markedly fewer tasks (41.2%). Inspecting its state objects shows why: because the schema is free-form, the agent writes whatever fields the current turn suggests, so what counts as a required slot drifts between turns and the agent has no fixed criterion for deciding when a task is complete. Externalising state is therefore not sufficient on its own; the gain in GSC-QA comes from state constrained by a declared goal and required-slot ontology against which completeness is well defined. This distinction is the substance of our claim, and neither a longer context window nor a checkpointers supplies it. The SimpleTOD-style baseline reinforces the same point from the other direction: despite explicitly emitting a belief state on every turn, it reaches only 39.0%, because that state lives in the generated sequence rather than in an external store.

To rule out judge dependence we re-scored every arm under four judge families drawn from four independent providers (Table 14). GSC-QA ranks first under all four. Its advantage is significant under every one of them over the SimpleTOD-style model, the graph agent, and the two vanilla backbones, which trail by 7.7 to 47.0 points (McNemar $p < 0.05$ in all sixteen of those cells). Over the two instruction-conditioned baselines it leads under all four judges as well, significantly under *gpt-5.4* and *qwen3.7-plus* but not under *gemini-3-flash*, which compresses all strong systems toward its ceiling, nor under *minimax-m3*. No baseline beats GSC-QA anywhere. This is the form in which we ask the reader to take the comparison: a ranking that four independent evaluators agree on, rather than a margin measured by one.

Table 14 gives the same result as absolute levels rather than as differences.

GSC-QA ranks highest under all four judges, and its advantage over SimpleTOD, the graph agent and every non-instructed baseline is significant under each of them.

Over the strongest instruction-conditioned baselines the margin, though never reversed, is judge-sensitive: it is significant under *gpt-5.4* and *qwen3.7-plus* but not under *gemini-3-flash*, which rates every strong system near ceiling ($\approx 88\%$) and so does not separate them. The ordering is therefore judge-robust; only the size of the margin over the strongest baselines depends on who is judging.

M. GENERALIZATION TO A PUBLIC BENCHMARK

To test whether GSC-QA's core mechanism (persistent, cross-turn goal and slot tracking) generalizes beyond our Vietnamese enterprise data, we evaluate it on the public English benchmark MultiWOZ 2.2 [11], scoring against its gold belief-state annotations so that evaluation is fully objective and uses *no* LLM judge. On a 284-turn test slice we compare two *gpt-4.1-mini* dialogue-state trackers on the same backbone: a *goal-tracking* tracker that maintains and re-emits the complete cumulative belief state from the full history (mirroring GSC-QA's persistent store), and a *stateless* tracker that extracts slots from the current user turn only (mirroring a single-turn baseline). We report Joint Goal Accuracy (JGA, exact set match of the belief state) and slot F1 (Table 15). The goal-tracking variant more than doubles JGA (20.1% vs. 8.8%) and triples slot F1 (0.635 vs. 0.205); the gap is driven almost entirely by recall (0.651 vs. 0.124), as the stateless tracker systematically loses slots established in earlier turns, precisely the failure mode persistent goal tracking is designed to prevent. Absolute JGA is modest because both are *zero-shot* LLM trackers given only the active domain names, not fine-tuned MultiWOZ DST models, so we make no state-of-the-art claim; the relevant result is the large, objective, judge-independent gap between persistent and stateless tracking, confirming that the mechanism transfers to public data.

N. LLM BACKBONE COMPARISON

To assess sensitivity of the GSC-QA framework to the choice of generation backbone, we evaluate the full GSC-QA pipeline with three LLM configurations across the 835-record dataset: one cloud-hosted OpenAI model (*gpt-4.1-mini*) and two on-premise open-source models served via Ollama (*qwen3.5:27b*, *gpt-oss:20b*) for privacy-sensitive

TABLE 15. Dialogue-state tracking on a MultiWOZ 2.2 test slice (284 user turns), scored against gold belief states (no LLM judge). Both trackers are zero-shot `gpt-4.1-mini`; the goal-tracking variant emulates GSC-QA's persistent cross-turn state.

Tracker	JGA (%) ↑	slot-P ↑	slot-R ↑	slot-F1 ↑
Stateless (single-turn)	8.8	0.580	0.124	0.205
Goal-tracking (persistent)	20.1	0.619	0.651	0.635

TABLE 16. GSC-QA performance under different LLM backbones. Cloud models use the OpenAI API; local models are served via Ollama on an NVIDIA RTX ADA 5880 (48 GB). Best per column is bolded. [‡]ERR re-evaluated on the organic pivot set ($n = 205$, no synthetic data) from responses re-generated by each backbone, which is why the `gpt-4.1-mini` value here (70.7%, 145/205) is below the 79.5% of Table 4 (163/205): that figure scores the deployed system's logged production output on the same records, under the same judge and rubric. Only this table's column is comparable across backbones.

LLM Backbone	TSR (%)	GCS	ERR [‡] (%)
<i>Cloud-hosted (OpenAI API)</i>			
<code>gpt-4.1-mini</code>	87.1	0.784	70.7
<i>Local (Ollama, NVIDIA RTX ADA 5880 48 GB)</i>			
<code>qwen3.5:27b</code>	84.3	0.762	80.5
<code>gpt-oss:20b</code>	83.2	0.758	40.0

enterprise environments. All components (retrieval, goal tracking, expert instructions) remain identical; only the generation backbone changes. Table 16 summarises the results.

Several observations follow. Note first that the ERR values in this table are measured on *re-generated* responses: each backbone is swapped into the pipeline and re-run over the 205 organic pivot records under an identical judge, so the `gpt-4.1-mini` figure here (70.7%, 145/205) is not the production-trace ERR of Table 4 (79.5%, 163/205), which reflects the deployed system's logged output. First, `gpt-4.1-mini` achieves 87.1% TSR and 70.7% ERR under this controlled re-run, confirming that the framework's performance is primarily driven by the goal-tracking and instruction-injection architecture rather than raw model capability. Second, `qwen3.5:27b` achieves the highest ERR across all backbones (80.5%), surpassing both cloud models; its TSR (84.3%) and GCS (0.762) remain competitive with a 2.8-point TSR gap relative to `gpt-4.1-mini`, likely reflecting stronger Vietnamese instruction adherence in this model's training corpus. Third, `gpt-oss:20b` (83.2% TSR, GCS 0.758) exhibits substantially lower ERR (40.0%), indicating that while its TSR is adequate for transactional goals, the model struggles to reliably re-anchor to the original goal after off-topic pivots, a limitation attributed to reduced Vietnamese conversational coherence in the 20B-parameter quantized configuration. These results suggest that ERR is more sensitive to backbone capability than TSR or GCS, and that model selection for pivot-heavy deployments should prioritise Vietnamese conversational robustness. For deployments requiring data sovereignty, `qwen3.5:27b` represents the strongest on-premise option, accepting a

TABLE 17. End-to-end per-turn latency distribution on the cloud backbone (`gpt-4.1-mini`), measured over 247 production traces. Values in seconds.

Component	Mean	Std	p50	p95	p99	Max
Total pipeline	2.68	3.00	2.53	4.41	12.50	33.76
LLM call	2.34	1.04	2.14	3.19	10.49	10.79

2.8-point TSR trade-off in exchange for superior pivot recovery (80.5% vs 70.7%). Fig. 3 visualizes the trade-off between latency and task accuracy across backbones.

O. SCALABILITY AND DEPLOYMENT COST

Deploying GSC-QA for a new enterprise vertical requires three operator-side steps: (1) authoring expert instruction documents for each goal category (4 hours per goal set of 9 goals, including domain review); (2) configuring the required-slot schema in the goal specification database; and (3) registering intent descriptions for the two-stage classifier. Steps (1)–(3) require no model retraining and can be completed by a non-ML domain operator. For automated instruction generation, operator-provided Standard Operating Procedure (SOP) documents can be parsed by an LLM to produce initial Q&A rule drafts, reducing manual authoring time by an estimated 60%. In production, the full GSC-QA pipeline processes each user turn in 2.68 s end-to-end (cloud) or 4.18 s (on-premise, `gpt-oss:20b`), within the sub-5-second latency acceptable for text-based enterprise chat interfaces.

Because averages hide tail behaviour that matters for deployment, Table 17 reports the full end-to-end latency distribution measured over 247 production turn-level traces on the cloud backbone (`gpt-4.1-mini`). The median (2.53 s) is close to the reported mean, but the distribution is right-skewed: the 95th percentile is 4.41 s and the 99th percentile reaches 12.50 s, with a rare worst case of 33.76 s. The LLM generation call accounts for most of the latency (median 2.14 s) and is more tightly bounded (p95 3.19 s); the heavy upper tail of the total is driven by occasional tool-call retries and slow backend API responses rather than by the retrieval or goal-tracking layers. For latency-sensitive deployments this motivates a timeout with graceful fallback, which we apply in production at 15 s on the LLM generation call rather than on the end-to-end turn; the end-to-end maximum of 33.76 s therefore reflects turns in which retries and slow backend API responses accumulated outside that call.

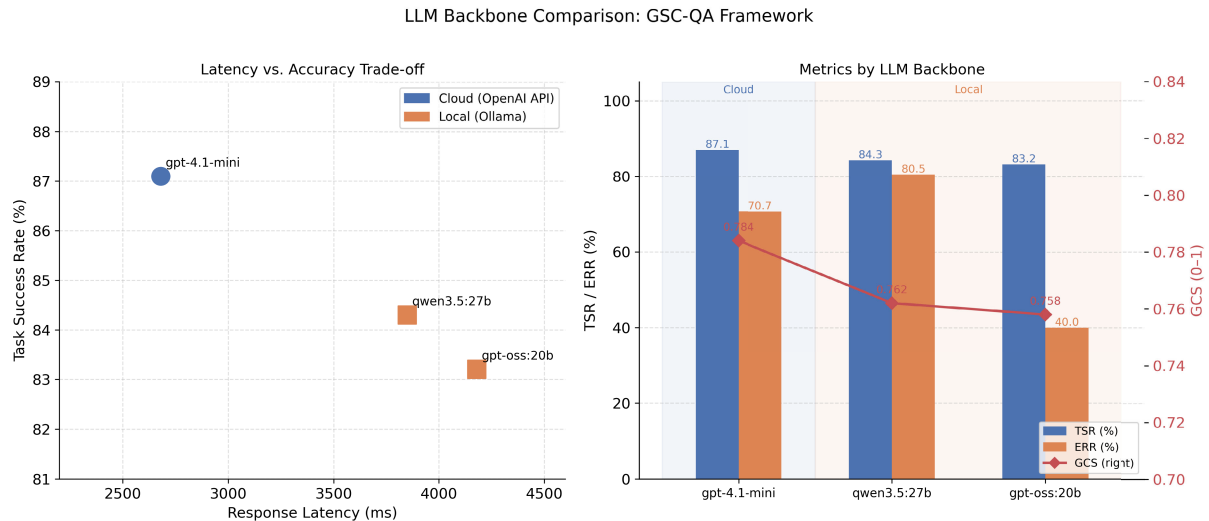


FIGURE 3. LLM backbone comparison: Accuracy (TSR) vs. Latency. Cloud models offer superior efficiency, while local models (qwen3.5:27b) provide competitive accuracy for private deployments.

P. VIETNAMESE LANGUAGE CONSIDERATIONS

Deploying GSC-QA for a new enterprise vertical requires three operator-side steps: (1) authoring expert instruction documents for each goal category (~ 4 hours per goal set of 9 goals, including domain review); (2) configuring the required-slot schema in the goal specification database; and (3) registering intent descriptions for the two-stage classifier. Steps (1)–(3) require no model retraining and can be completed by a non-ML domain operator. For automated instruction generation, operator-provided Standard Operating Procedure (SOP) documents can be parsed by an LLM to produce initial Q&A rule drafts, reducing manual authoring time by an estimated 60%. In production, the full GSC-QA pipeline processes each user turn in 2.68 s end-to-end (cloud) or 4.18 s (on-premise, gpt-oss:20b), within the sub-5-second latency acceptable for text-based enterprise chat interfaces.

Q. LIMITATIONS

This study has several limitations. First, while the evaluation spans ten enterprise domains and 1,516 records, all data originates from Vietnamese-language deployments within a single provider ecosystem; generalization to other languages, and external validity to independently-built dialogue systems, requires further validation; we partially address this with a zero-shot dialogue-state-tracking result on the public MultiWOZ 2.2 benchmark (Section IV-M), but a full evaluation of the deployed system on such benchmarks, and a Vietnamese-language public benchmark, remains for future work. Second, the pivot evaluation uses exclusively organic pivot scenarios ($n = 205$); future work should extend coverage across more enterprise verticals. Third, expert instructions are currently manually curated; automated instruction generation from policy documents represents a promising direction for reducing operator

effort in future deployments. Fourth, the LLM comparison (Section IV-N) evaluates only three backbone configurations (gpt-4.1-mini, qwen3.5:27b, gpt-oss:20b); a more exhaustive sweep across model families and quantization schemes is left for future work. Fifth, while the human evaluation validates the LLM-as-judge approach with strong inter-annotator agreement ($\kappa = 0.81$) and correlation ($r = 0.89$), the 100-record sample may not capture all edge cases, and future work should extend human annotation to the full dataset.

Finally, and most consequentially for how the results should be read, this work evaluates GSC-QA *task-completion mechanics* and not *user experience*. Every metric reported here (TSR, GCS, ERR, DE, JGA) is scored either against authored goal specifications or by an LLM judge; none of them reflects a signal originating from the end user. We collect no satisfaction rating, no thumbs-up/down feedback, no human-agent escalation rate, and no return-visit or retention measure, and the deployment logs available to us do not record escalation events, so no such proxy could be reconstructed retrospectively. The consequence is that our central design premise, that proactively steering a user toward the missing slots of an open goal is preferable to answering passively, is validated only in the sense that it completes more tasks, not in the sense that users prefer it. The opposite is entirely plausible for some interactions: an agent that persistently re-asks for outstanding slots may be experienced as intrusive, particularly on goals requiring three or more slots, and the latency tail documented in Table 17 may compound that effect. This also bears on the abandonment analysis above, since abandonment is precisely where task-completion metrics and user tolerance would diverge. Establishing whether proactive elicitation improves or degrades the interaction as experienced requires instrumented user-side signals or a controlled study with human participants, which we regard as the most important extension

of this work; claims in this paper should accordingly be read as claims about task completion alone. A further consideration is that TSR and ERR are computed against per-goal specifications and expert instructions, following the action- and policy-oriented evaluation tradition of task-oriented dialogue (e.g., ABCD [27]); their absolute values consequently depend on the authored goal specifications, so cross-system numerical comparison should be made with care. We mitigate this by documenting the specification template, which is available from the corresponding author on request, and by quantifying the low operator cost of authoring it (Section IV), and by additionally reporting the specification-agnostic GCS and Answer Relevancy judgements, which offer a more portable signal across independently-specified systems.

V. CONCLUSION

In this study, we present GSC-QA, a goal-directed sequential dialogue framework that integrates persistent goal tracking, expert instructions, and proactive slot management into RAG-based conversational systems. Experiments on 1,516 annotated records spanning ten Vietnamese enterprise domains (principally beauty services, professional services, fashion and retail, spa and wellness, and language education, with smaller deployments in marketing, healthcare, banking, and lighting and electrical equipment) demonstrate that goal persistence and expert instructions are complementary: goal tracking provides multi-turn consistency (79.1% TSR without instructions), while expert instructions ensure procedural correctness (GCS of 0.78 measured via RAGAS-style LLM-as-judge with gpt-5.4). Together, they achieve 87.1% TSR with 98.6% instruction compliance, substantially outperforming both standard RAG (58.7% TSR, GCS 0.62) and the Instruct-Only baseline (63.2% TSR, GCS 0.63) evaluated on the same dataset. Per-goal analysis further confirms that goals with explicit procedural constraints (e.g., `collect_contact` at GCS 0.875) consistently outperform underspecified goals, validating the central role of the Instruction Construction Layer in the GSC-QA architecture.

Future work will address current limitations by extending GSC-QA to multilingual settings, automating expert instruction generation from policy documents, and broadening the LLM backbone comparison. Longer-term, integrating Reinforcement Learning from Human Feedback (RLHF) to optimize dialogue policy and transitioning to a distributed multi-agent architecture represent natural evolution paths for enterprise deployment.

ETHICAL CONSIDERATIONS

APPENDIX

LLM-AS-JUDGE EVALUATION PROMPTS

This appendix documents the exact system prompts used for the automated GCS (Goal Consistency Score) evaluation described in Section III. Evaluation was implemented via the OpenRouter API (`openai/gpt-5.4`) using gpt-5.4 as the judge model with `temperature=0` and structured JSON output (`response_format: json_object`) to

ensure deterministic scoring. Each prompt was applied in batches of 10 records; scores were clipped to [0, 1] and averaged per goal category.

A. FAITHFULNESS PROMPT (GCS PROXY)

The faithfulness prompt measures whether the system response is grounded in and consistent with the expert instruction context, serving as the primary proxy for Goal Consistency Score (GCS).

[System] :

You are a chatbot evaluation expert.
Task: assess whether the ANSWER is "faithful" (grounded, non-fabricated) with respect to the CONTEXT.
The CONTEXT is the system prompt containing the bot's expert instructions.

Scoring rules:

- Score 1.0: Answer fully follows the context; no fabricated information
- Score 0.7: Answer is mostly consistent but mentions details not covered in the context
- Score 0.5: Answer partially fabricates or partially contradicts the expert instructions
- Score 0.0: Answer is entirely fabricated or clearly violates the expert instructions

Return a JSON object: `{"results": [{"score": 0.0-1.0, "reason": "brief reason"}, ...]}`
The number of results must equal the number of input items.

[User] :

`{"id": i, "context": <expert_instruction[:400]>, "answer": <response[:300]>} × batch_size`

B. ANSWER RELEVANCY PROMPT

The relevancy prompt measures whether the system response answers the user's question correctly and on-topic. This metric (*answer_relevancy* in the raw evaluation outputs) is reported for GSC-QA in Section IV-K (Table 12): 0.71 on informational (question-bearing) turns of the primary set, with the lower aggregate driven by slot-provision turns where the user supplies information rather than posing a question.

[System] :

You are a chatbot evaluation expert for enterprise advisory services.
Task: assess whether the ANSWER is relevant and correctly responds to

the QUESTION.

Scoring rules:

- Score 1.0: Answer is precise, complete, and directly on-topic
- Score 0.7: Answer is relevant but lacks some information or is slightly off-target
- Score 0.5: Answer partially addresses the question or redirects excessively
- Score 0.0: Answer is irrelevant or entirely off-topic

Return a JSON object: {"results": [{"score": 0.0-1.0, "reason": "brief reason"}, ...]}

The number of results must equal the number of input items.

[User]:

```
{ "id": i, "question":
<user_utterance_input>,
"answer": <response[:300]> }
x batch_size
```

C. IMPLEMENTATION DETAILS

- **Judge model:** gpt-5.4 accessed via the OpenRouter API (model_id: openai/gpt-5.4).
- **Inference settings:** temperature=0.
- **Batch size:** 10 records per API call to reduce latency while remaining within context limits.
- **Context truncation:** Expert instruction context truncated to 400 characters; response truncated to 300 characters per item to fit batch within model context.
- **Retry policy:** Up to 2 retries with 1.5 s back-off on API failure; fallback score 0.5 assigned on persistent failure.
- **Score clipping:** All scores clipped to [0.0, 1.0]; non-numeric responses replaced with fallback 0.5.
- **GCS computation:** The GCS reported in the paper corresponds to the mean faithfulness score across all evaluated records within each system configuration.

ACKNOWLEDGMENT

During the preparation of this work, the author(s) used Claude AI and Grammarly to improve the manuscript's readability and language. After using this tool/service, they reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

REFERENCES

- [1] M. Arslan, S. Munawar, Z. Riaz, and C. Cruz, "Large language models for business and management applications: A review," *Inf. Process. Manage.*, vol. 63, no. 7, Nov. 2026, Art. no. 104864. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457326002554>
- [2] P. Li, J. Wang, Q. Yang, T. Guo, Y. Liu, and Y. Wen, "From answering to discussing: Advancing human-AI cognitive collaboration in dialogue agents," *Inf. Process. Manage.*, vol. 63, no. 5, Jul. 2026, Art. no. 104711. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457326001020>
- [3] S. Farshidi, K. Rezaee, S. Mazaheri, A. H. Rahimi, A. Dadashzadeh, M. Ziabakhsh, S. Eskandari, and S. Jansen, "Understanding user intent modeling for conversational recommender systems: A systematic literature review," *User Model. User-Adapted Interact.*, vol. 34, no. 5, pp. 1643–1706, Jun. 2024.
- [4] J. Song, Z. Ashktorab, Q. Pan, C. Dugan, W. Geyer, and T. W. Malone, "Interaction configurations and prompt guidance in conversational AI for question answering in human-AI teams," *Proc. ACM Hum.-Comput. Interact.*, vol. 9, no. 7, pp. 1–27, Oct. 2025.
- [5] Y. Deng, W. Lei, W. Lam, and T.-S. Chua, "A survey on proactive dialogue systems: Problems, methods, and prospects," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, Aug. 2023, pp. 6583–6591, doi: [10.24963/ijcai.2023/738](https://doi.org/10.24963/ijcai.2023/738).
- [6] K. Alushi, J. Strich, C. Biemann, and M. Semmann, "Comprehensive comparison of RAG methods across multi-domain conversational QA," in *Proc. 19th Conf. Eur. Chapter Assoc. Comput. Linguistics*, Mar. 2026, pp. 233–247. [Online]. Available: <https://aclanthology.org/2026.eacl-srw.17/>
- [7] Z. Liu, W. Ping, R. Roy, P. Xu, C. Lee, M. Shoeybi, and B. Catanzaro, "ChatQA: Surpassing GPT-4 on conversational QA and RAG," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2024, pp. 15416–15459.
- [8] J. Xiao, W. Wu, J. Zhao, M. Fang, and J. Wang, "Enhancing long-form question answering via reflection with question decomposition," *Inf. Process. Manage.*, vol. 62, no. 6, Nov. 2025, Art. no. 104274. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457325002158>
- [9] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. Eur. Chapter Assoc. Comput. Linguistics, Syst. Demonstrations*, Mar. 2024, pp. 150–158. [Online]. Available: <https://aclanthology.org/2024.eacl-demo.16/>
- [10] Z. Ding, Z. Yang, and H. Lin, "A plug-and-play adapter for consistency identification in task-oriented dialogue systems," *Inf. Process. Manage.*, vol. 61, no. 3, May 2024, Art. no. 103637. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457323003746>
- [11] P. Budzianowski, T.-H. Wen, B.-H. Tseng, I. Casanueva, S. Ultes, O. Ramadan, and M. Gašić, "MultiWOZ—A large-scale multi-domain Wizard-of-Oz dataset for task-oriented dialogue modelling," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Oct. 2018, pp. 5016–5026. [Online]. Available: <https://aclanthology.org/D18-1547/>
- [12] C.-S. Wu, A. Madotto, E. Hosseini-Asl, C. Xiong, R. Socher, and P. Fung, "Transferable multi-domain state generator for task-oriented dialogue systems," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, Jul. 2019, pp. 808–819. [Online]. Available: <https://aclanthology.org/P19-1078/>
- [13] E. Hosseini-Asl, B. McCann, C.-S. Wu, S. Yavuz, and R. Socher, "A simple language model for task-oriented dialogue," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 20179–20191.
- [14] Y. Wang, Z. Wu, J. Yao, and J. Su, "TDAG: A multi-agent framework based on dynamic task decomposition and agent generation," *Neural Netw.*, vol. 185, May 2025, Art. no. 107200. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0893608025000796>
- [15] M. Ahmed, H. U. Khan, and E. U. Munir, "Conversational AI: An explication of few-shot learning problem in transformers-based chatbot systems," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 2, pp. 1888–1906, Apr. 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10153715>
- [16] T. Han, X. Liu, R. Takanobu, Y. Lian, C. Huang, D. Wan, W. Peng, and M. Huang, "MultiWOZ 2.3: A multi-domain task-oriented dialogue dataset enhanced with annotation corrections and co-reference annotation," in *Proc. Natural Lang. Process. Chin. Comput. (NLPCC)*, 2020, pp. 206–218.
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 9459–9474.
- [18] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Proc. Findings Assoc. Comput. Linguistics: EMNLP*, Nov. 2021, pp. 3784–3803. [Online]. Available: <https://aclanthology.org/2021.findings-emnlp.320/>
- [19] R. Saha Roy, J. Schlotthauer, C. Hinze, A. Foltyn, L. Hahn, and F. Kuech, "Evidence contextualization and counterfactual attribution for conversational QA over heterogeneous data with RAG systems," in *Proc. 18th ACM Int. Conf. Web Search Data Mining*, Mar. 2025, pp. 1040–1043. [Online]. Available: <https://dl.acm.org/doi/10.1145/3701551.3704126>

- [20] N. Roy, L. F. R. Ribeiro, R. Bliloshmi, and K. Small, "Learning when to retrieve, what to rewrite, and how to respond in conversational QA," in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, Nov. 2024, pp. 10604–10625. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.622/>
- [21] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 68539–68551.
- [22] Y. Qin et al., "Tool learning with foundation models," *ACM Comput. Surveys*, vol. 57, no. 4, pp. 1–40, Dec. 2024, doi: [10.1145/3704435](https://doi.org/10.1145/3704435).
- [23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 1–11.
- [24] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 8634–8652.
- [25] C.-W. Liu, R. Lowe, I. Serban, M. Noseworthy, L. Charlin, and J. Pineau, "How NOT to evaluate your dialogue system: An empirical study of unsupervised evaluation metrics for dialogue response generation," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Nov. 2016, pp. 2122–2132. [Online]. Available: <https://aclanthology.org/D16-1230/>
- [26] S. Mehri and M. Eskenazi, "USR: An unsupervised and reference free evaluation metric for dialog generation," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics*, Jul. 2020, pp. 681–707. [Online]. Available: <https://aclanthology.org/2020.acl-main.64/>
- [27] D. Chen, H. Chen, Y. Yang, A. Lin, and Z. Yu, "Action-based conversations dataset: A corpus for building more in-depth task-oriented dialogue systems," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol.*, Jun. 2021, pp. 3002–3017. [Online]. Available: <https://aclanthology.org/2021.naacl-main.239/>
- [28] S. Larsson and D. R. Traum, "Information state and dialogue management in the TRINDI dialogue move engine toolkit," *Natural Lang. Eng.*, vol. 6, nos. 3–4, pp. 323–340, Sep. 2000.
- [29] D. Bohus and A. I. Rudnicky, "The RavenClaw dialog management framework: Architecture and systems," *Comput. Speech Lang.*, vol. 23, no. 3, pp. 332–361, Jul. 2009.
- [30] J. D. Williams and S. Young, "Partially observable Markov decision processes for spoken dialog systems," *Comput. Speech Lang.*, vol. 21, no. 2, pp. 393–422, Apr. 2007.
- [31] A. Rastogi, X. Zang, S. Sunkara, R. Gupta, and P. Khaitan, "Towards scalable multi-domain conversational agents: The schema-guided dialogue dataset," in *Proc. AAAI Conf. Artif. Intell.*, 2020, vol. 34, no. 5, pp. 8689–8696.
- [32] J. Andreas et al., "Task-oriented dialogue as dataflow synthesis," *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 556–571, Jul. 2020.
- [33] B. Peng, C. Li, J. Li, S. Shayandeh, L. Liden, and J. Gao, "Soloist: Building task bots at scale with transfer learning and machine teaching," *Trans. Assoc. Comput. Linguistics*, vol. 9, pp. 807–824, Aug. 2021.
- [34] Y. Yang, Y. Li, and X. Quan, "UBAR: Towards fully end-to-end task-oriented dialog system with GPT-2," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 16, pp. 14230–14238.
- [35] Y. Su, L. Shu, E. Mansimov, A. Gupta, D. Cai, Y.-A. Lai, and Y. Zhang, "Multi-task pre-training for plug-and-play task-oriented dialogue system," in *Proc. 60th Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 4661–4676.
- [36] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," 2023, *arXiv:2310.08560*.
- [37] LangChain AI. (2024). *LangGraph: A Library for Building Stateful, Multi-Actor Applications With LLMs*. Accessed: 2026. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [38] Anthropic. (2024). *Model Context Protocol Specification*. [Online]. Available: <https://modelcontextprotocol.io>
- [39] S. Reddy, D. Chen, and C. D. Manning, "CoQA: A conversational question answering challenge," *Trans. Assoc. Comput. Linguistics*, vol. 7, pp. 249–266, Nov. 2019.
- [40] E. Choi, H. He, M. Iyyer, M. Yatskar, W.-T. Yih, Y. Choi, P. Liang, and L. Zettlemoyer, "QuAC: Question answering in context," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2018, pp. 2174–2184.
- [41] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
- [42] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," *ACM Trans. Softw. Eng. Methodology*, Feb. 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3796519>
- [43] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers Comput. Sci.*, vol. 18, no. 6, Mar. 2024, Art. no. 186345, doi: [10.1007/s11704-024-40231-1](https://doi.org/10.1007/s11704-024-40231-1).



QUOC-DUNG NGO received the M.Sc. degree in knowledge management and decision making from Polytech Nantes–Lyon 2 and the Ph.D degree in applied computer science and automation from INP Grenoble, France. He is currently an Associate Professor with the Research Institute of Posts and Telecommunications, Posts and Telecommunication Institute of Technology, Hanoi, Vietnam. His research interests include computer security, energy monitoring, and automation systems.



QUANG-DUNG DANG received the Engineering degree in information technology from the Posts and Telecommunications Institute of Technology, Hanoi, Vietnam, in 2024, where he is currently pursuing the master's degree with the Research Institute of Posts and Telecommunications. His research interests include artificial intelligence, generative AI, and data mining.



LY-HUYNH PHAN received the master's degree in data science from INP Grenoble, France, and the Ph.D. degree in applied computer science (mechanical) from the University of Bordeaux, France, in 2022. He is currently a Lecturer with the Research Institute of Posts and Telecommunications, Posts and Telecommunication Institute of Technology, Hanoi, Vietnam. His research interests include artificial intelligence, energy modeling, smart building, and energy efficiency.

...